

Software Engineering Group Project

Design Document

Author: Natalia Spence (nas97), Niall Perry (nip33),
Francesca Sasso (frs30), Reema Fraser (ref17),
Abdullah Al-Samarrai (aba115), Joe Ball (job130),
William Shipp (wis40), Prem Sharma (prs49)

Date: 26 March 2025

Version: 1.2

Status: Release

Department of Computer Science,
Aberystwyth University,
Aberystwyth,
Ceredigion, SY23 3DB,
U.K.

CONTENTS

1	INTRODUCTION	2
1.1	Purpose of this Document	2
1.2	Scope	2
1.3	Objectives	2
2	STATIC ARCHITECTURE DESCRIPTION	3
2.1	Programs in the System and their Deployment	3
2.1.1	Systems Diagram	3
2.1.2	Systems Architecture	3
2.2	Package Diagram	4
2.2.1	Package Architecture	4
3	STATIC DESCRIPTION	5
3.1	Class Diagrams	5
3.1.1	Screen	6
3.1.2	Title	6
3.1.3	Customization	6
3.1.4	CardFrontCustomization	7
3.1.5	Table	8
3.1.6	Popup	9
3.1.7	Replay	9
3.1.8	TableObject	10
3.1.9	Suit	11
3.1.10	Card	11
3.1.11	Deck	12
3.1.12	GameOptions	14
3.1.13	ActionType	15
3.1.14	GameAction	15
3.1.15	Recording	15
3.1.16	SoundManager	16
3.2	Discussion of Key Data Structures	16
3.2.1	Player Customization	16
3.2.2	Game Recording Data	16
3.3	Mapping from User Stories to Classes	17
4	DYNAMIC DESCRIPTION	18
4.1	Sequence Diagrams and Pseudocode	18
4.2	State diagram	45
	REFERENCES	47
	DOCUMENT HISTORY	48

1 INTRODUCTION

1.1 Purpose of this Document

This document aims to provide a comprehensive overview of the design and architecture of Group 06's Deck of Cards project, developed for the Software Engineering module at Aberystwyth University in 2025.

1.2 Scope

The document details the system architecture, including package breakdowns, system interactions, and visual representations such as package, class, and sequence diagrams. Additionally, it contains drafts of pseudocode for complex functions, the majority of which can also be found in GitLab issue comments. While prior familiarity with the DS1 plan [1] is not required, it is recommended for additional context. However, this document assumes that the reader is knowledgeable about the SE.QA.05 [2] Design Standards and the Product Backlog [3].

1.3 Objectives

The primary objective of this document is to clearly present the system's architecture, package structure, and key pseudocode components. It aims to illustrate how the various system components interact and justify the chosen package structure to enhance code readability and maintainability. This document serves as a foundation for developing a system that is both adaptable and well-structured.

2 STATIC ARCHITECTURE DESCRIPTION

2.1 Programs in the System and their Deployment

2.1.1 Systems Diagram

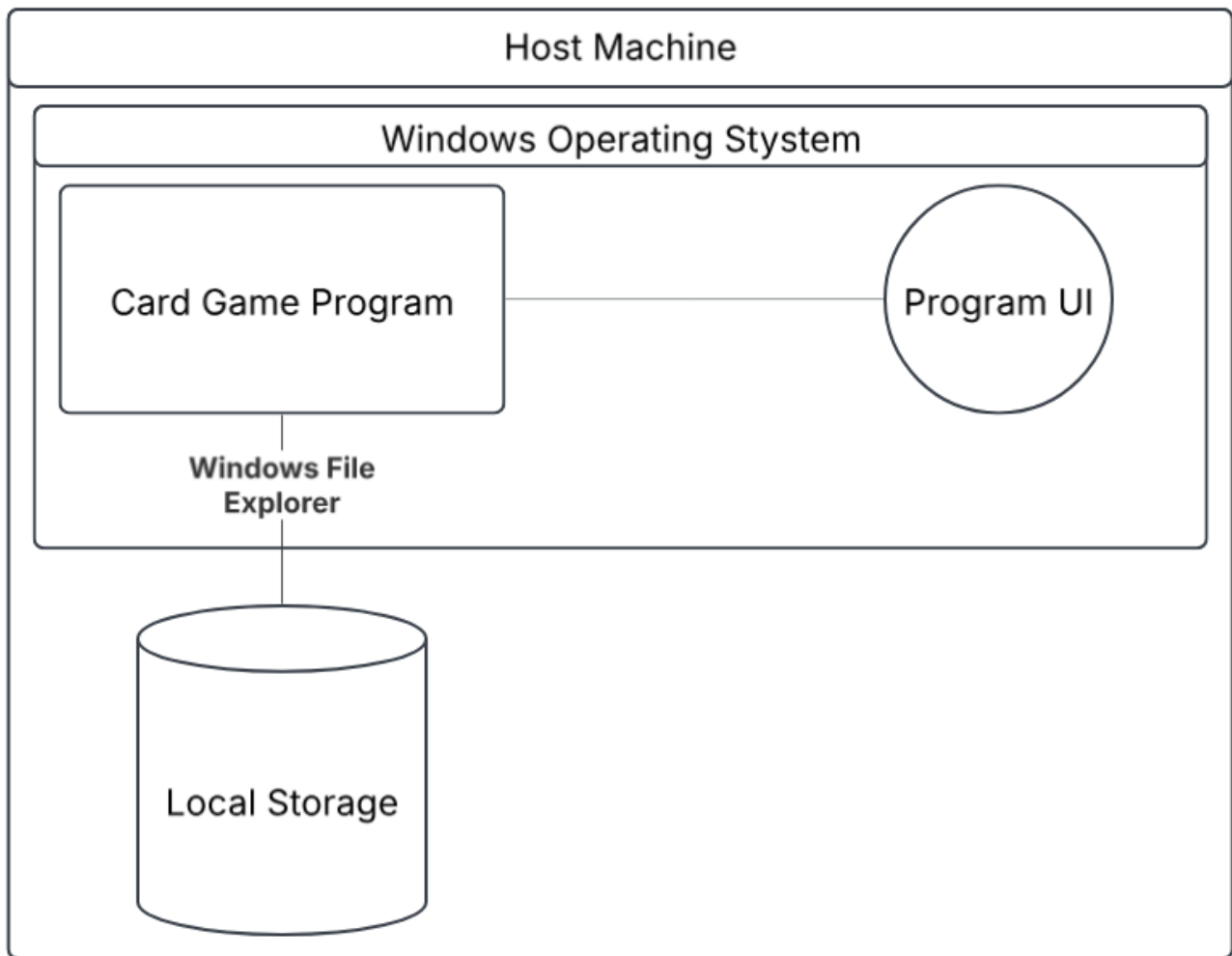


Figure 1: The Systems Diagram

2.1.2 Systems Architecture

The system architecture comprises three primary components that interact on the host machine to execute the program: the program engine, the user interface (UI), and local storage.

The program engine is responsible for executing the card game's core logic. It manages game mechanics, processes user inputs, and controls the overall flow of the application. The engine also integrates with JavaFX, which facilitates the user interface UI — the second key component. The UI provides a graphical representation through which users can interact with the program, enabling input and engagement with the game.

The third component is the local storage of the host machine, which is used to store essential assets such as card and table images, as well as game and dealing layout recordings. The program engine accesses this storage through the host operating system's built-in file management capabilities, such as Windows File Explorer for Windows.

All three components operate locally on a single host machine, with no requirement for external network access to ensure full program functionality.

2.2 Package Diagram

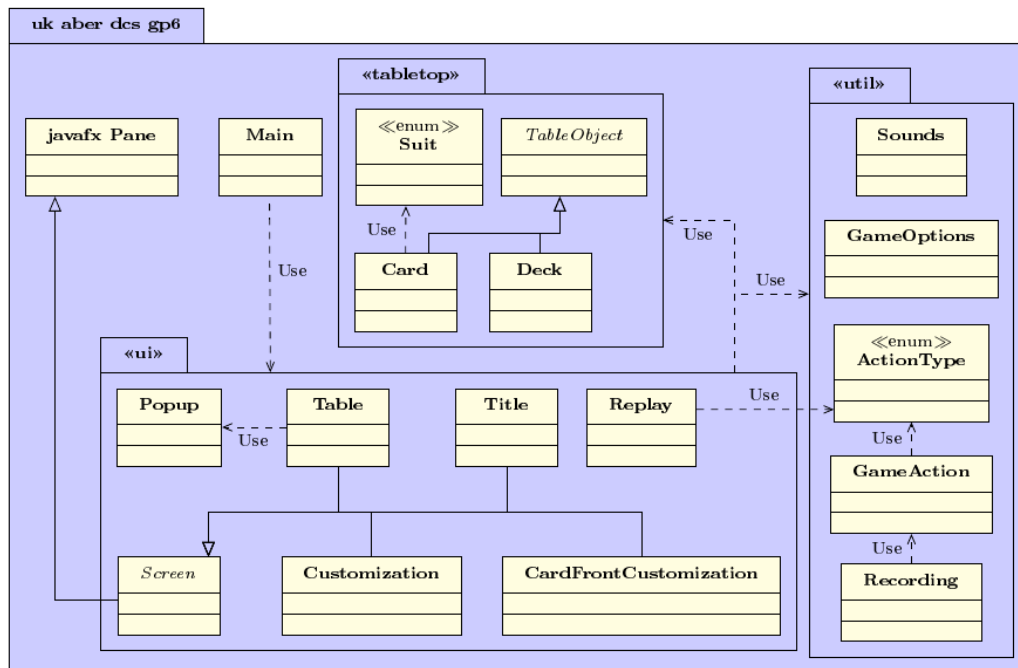


Figure 2: The Package Diagram

2.2.1 Package Architecture

The packages in the design of the program are split into three groups:

- 1) *ui* - Manages the user interface, including displaying contents on the window and receiving user input.
- 2) *tabletop* - The data structures that ensure the program can actually function, including the cards and decks.
- 3) *util* - Various other utility classes used for other purposes, such as recording games, playing sound effects or saving files for custom images. These essentially act as controllers for everything that isn't ui or data structure related.

Package Interactions

The *ui* package is the most prominent aspect of the application, with every screen involved in the program being contained within. All display and user interaction occurs within the *ui* package, which in turn communicates with both other packages to cause changes in the system, and the content displayed by the *ui* package.

The *tabletop* package is the core of the system, where the actual data structures are located, and is often used by the *ui* package to allow the program to function. Without it, you would have a window but the core program itself would not exist. Any direct manipulation of these data structures (aside from their positions on the UI) is located here.

Finally, the *util* package contains the other useful classes required for the program to operate. The *ui* package will regularly interact with the *util* package for important functionality that's not directly occurring on the UI, such as recording games, playing sound effects, saving user settings and loading user images.

3 STATIC DESCRIPTION

3.1 Class Diagrams

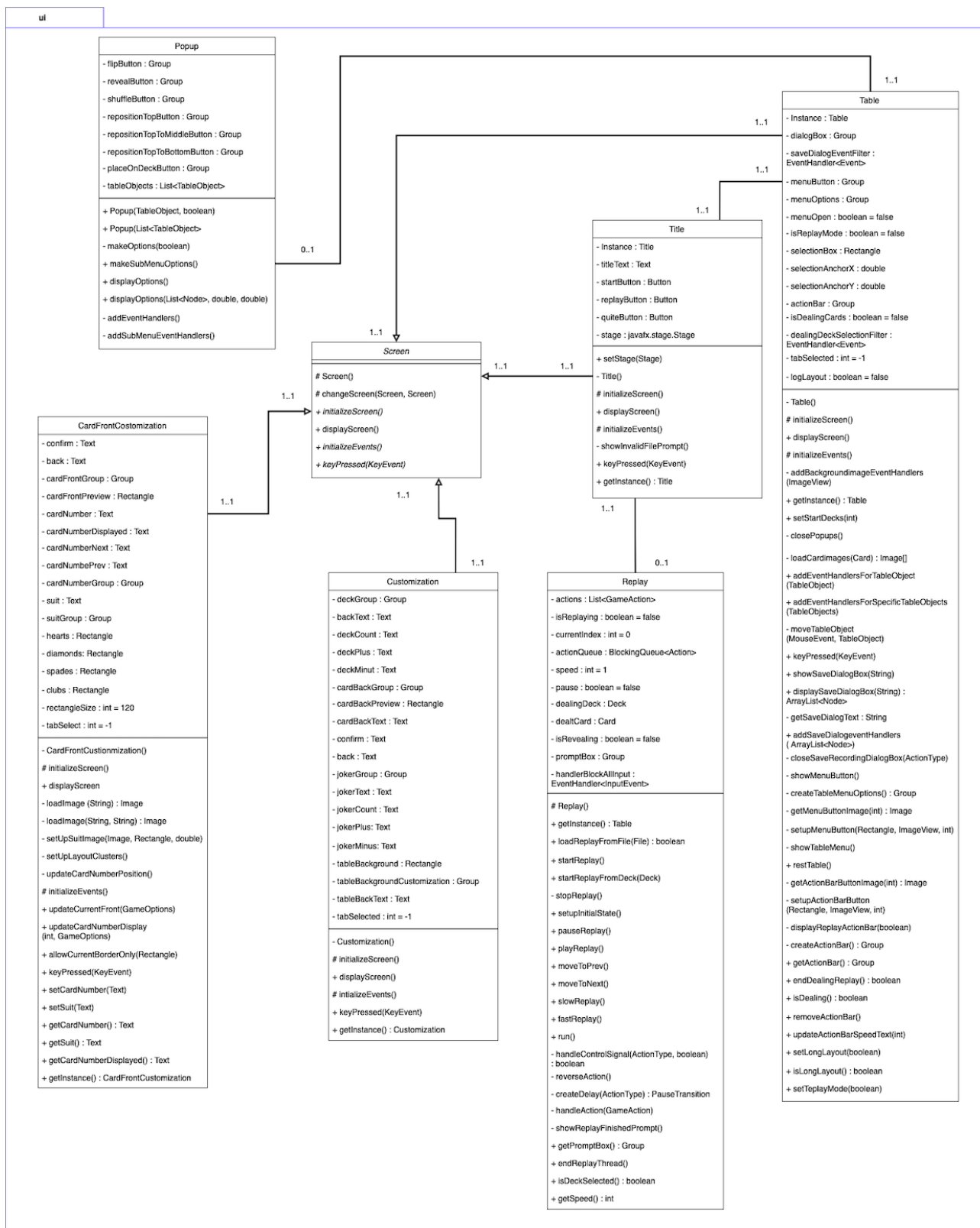


Figure 3: The Class Diagram for the ui Package

3.1.1 Screen

Screen is an abstract base class for all UI screens in the game. It defines the essential methods each screen must implement, such as initializing UI elements, displaying the screen, handling events, and reacting to key inputs. It also includes a helper method to switch between screens. Here are some of its functions:

- `Screen()`: Constructor that initializes a screen by calling abstract methods `initializeScreen()` and `initializeEvents()` to set up UI elements and event handlers.
- `changeScreen(Screen current, Screen next)`: Switches the current screen to a different one. If moving from `CardFrontCustomization` to `Table`, it also sets the number of decks before displaying the new screen.
- `initializeScreen()` (abstract): Must be implemented by subclasses to add and configure UI components for the screen.
- `displayScreen()` (abstract): Called every time the screen is shown. Subclasses use this to layout and update UI components based on window size.
- `initializeEvents()` (abstract): Subclasses implement this method to register event listeners and filters, such as mouse or button actions.
- `keyPressed(KeyEvent keyEvent)` (abstract): Handles keyboard input specific to the screen (e.g., tab navigation, confirmation via Enter).

3.1.2 Title

The *Title* class is rather small and simply handles the title screen.

It has a small number of attributes and methods, the former are listed below:

- `Instance`: the instance of the Title screen
- `titleText`: the program name text displayed at the top of the screen
- `startButton`: the button to go into the Customization screen to start the game
- `replayButton`: the button to open a file chooser to play a replay file
- `quitButton`: the button to quit the program
- `stage`: the JavaFX stage, used for the replay file chooser

The methods involved are consistent with the rest of the screens:

- `Title`: the constructor, creates the title screen
- `setStage`: sets the JavaFX stage, used for the file chooser for replays
- `initializeScreen`: creates all components to be displayed on the screen
- `displayScreen`: actually displays the screen, setting the size and position of components
- `initializeEvents`: sets up all the event handlers required to interact with the screen
- `showInvalidFilePrompt`: shows a prompt with an error when an invalid replay file is selected
- `keyPressed`: handles keyboard input
- `getInstance`: gets the Title instance

3.1.3 Customization

The *Customization* class handles the user interface for setting up visual preferences.

- Allows users to select custom card and table images

- Lets users choose the number of decks and jokers
- Uses JavaFX UI elements for interactive customization
- Called before starting or replaying a game

3.1.4 CardFrontCustomization

The CardFrontCustomization class represents a user interface screen that allows players to customize the front face of their playing cards by selecting a card number, suit, and uploading custom images. It follows the singleton pattern to ensure only one instance exists, and it integrates mouse and keyboard interaction to facilitate navigation and customization.

The attributes of this class are:

- CardFrontCustomization(): Private constructor to enforce the singleton pattern. Ensures only one instance of this screen exists.
- initializeScreen(): Sets up all UI elements including card preview, suit icons, navigation buttons, and layout groups on the screen.
- displayScreen(): Configures the layout and appearance of the screen, initializes the card preview with default settings, and loads images for suits and the card front.
- loadImage(String path): Loads an image from a given resource path and returns it, or logs a warning and returns null if not found.
- loadImage(String primaryPath, String fallbackPath): Attempts to load an image from the primary path, falls back to a secondary path if the first fails.
- setUpSuitImage(Image suitImage, Rectangle suitShape, double xPos): Sets image and position for a suit rectangle (clubs, hearts, etc.) used in the UI.
- setUpLayoutClusters(): Binds UI element positions to the screen's size for responsive layout design.
- updateCardNumberPosition(): Re-centers the card number text between the navigation arrows based on current layout bounds.
- initializeEvents(): Attaches mouse click event handlers for suit selection, card image updates, navigation arrows, and screen navigation buttons.
- updateCurrentFront(GameOptions options): Updates the card front preview with the selected suit and number image, falling back to default if user-customized image isn't found.
- updateCardNumberDisplay(int difference, GameOptions options): Changes the current card number and updates its corresponding label (e.g., "Ace", "Jack", "10").
- allowCurrentBorderOnly(Rectangle obj): Highlights the selected suit by giving it a colored border and removes it from others.
- keyPressed(KeyEvent keyEvent): Handles keyboard navigation (Tab, Shift+Tab, Escape, and Enter) between UI elements on the screen.
- setCardNumber(Text text): Sets the internal cardNumber Text object.
- setSuit(Text text): Sets the internal suit Text object.
- getCardNumber(): Returns the current card number Text object.
- getSuit(): Returns the current suit Text object.
- getCardNumberDisplayed(): Returns the Text object displaying the card number label (e.g., "King").

- `getInstance()`: Provides access to the singleton instance of this class.

3.1.5 Table

Table is the core interactive UI screen in the card game that allows users to interact with cards and decks. It handles rendering, mouse/keyboard events, replay controls, pop-ups, layout management, and deck/card manipulation. It's implemented as a singleton and extends the abstract *Screen* class.

- `Table()`: Private constructor to enforce singleton pattern.
- `getInstance()`: Returns the single instance of the *Table*.
- `initializeScreen()`: Creates and adds all UI components including menu, action bar, and selection rectangle.
- `displayScreen()`: Renders all visual components, background image, menu options, dialog boxes, and handles replay UI states.
- `initializeEvents()`: Attaches all necessary mouse event handlers for object interaction and menu toggling.
- `keyPressed(KeyEvent)`: Handles keyboard shortcuts for moving, rotating, flipping, and manipulating selected objects or UI navigation.
- `setStartDecks(int deckCount)`: Adds and positions the specified number of decks to the table.
- `displayTableObject(TableObject)`: Adjusts the size, image, and layout of a Card or Deck on the table.
- `loadCardImages(Card)`: Loads customized or default front/back images for a given card.
- `addEventHandlersForTableObject(TableObject)`: Adds full interactivity (click, drag, rotate, hover) to a card or deck.
- `addEventHandlersForSpecificTableObject(TableObject)`: Adds specific handlers, e.g., right-drag to draw cards from decks.
- `moveTableObject(MouseEvent, TableObject)`: Handles dragging logic for *TableObjects* and overlap checks.
- `closePopups()`: Closes all open right-click popups on the table.
- `showSaveDialogBox(String)`: Displays a prompt for saving recordings or loading layouts.
- `displaySaveDialogBox(String)`: Constructs and renders a save/load dialog box.
- `getSaveDialogText()`: Returns the prompt text from the currently visible dialog.
- `addSaveDialogEventHandlers(ArrayList<Node>)`: Attaches logic to dialog box buttons.
- `closeSaveRecordingDialogBox(ActionType)`: Closes the dialog and resets state based on the recording type.
- `showMenuButton()`: Displays the hamburger-style menu button in the UI.
- `createTableMenuOptions()`: Creates menu options like recording toggle, grid, and exit.
- `getMenuButtonImage(int)`: Returns the image corresponding to a menu button number.
- `setupMenuButton(Rectangle, ImageView, int)`: Adds functionality to each menu button (e.g., exit app or toggle layout recording).
- `showTableMenu()`: Displays or hides the table's dropdown menu.
- `createActionBar()`: Builds the UI components (buttons and text) for the replay control bar.

- `getActionBar()`: Returns the action bar group.
- `displayReplayActionBar(boolean)`: Renders the replay controls (or prompts) based on the state.
- `getActionBarButtonImage(int)`: Provides images for replay buttons (e.g., play, pause, skip).
- `setupActionBarButton(Rectangle, ImageView, int)`: Assigns replay functions to buttons (play, pause, etc.).
- `removeActionBar()`: Removes the replay action bar from the screen.
- `updateActionBarSpeedText(int)`: Updates the replay speed display text.
- `resetTable()`: Resets and reinitializes the table (used after replay ends).
- `endDealingReplay()` / `isDealing()`: Ends or checks whether dealing layout replay is active.
- `setLongLayout(boolean)` / `isLongLayout()`: Flags if a dealing layout exceeds deck capacity.
- `setReplayMode(boolean)`: Sets replay mode state.

3.1.6 Popup

The *Popup* class generates menus for table objects when the user right-clicks a card or group.

- Displays options based on selected object(s) (e.g., flip, reveal, rotate)
- Built using JavaFX Group and Rectangle components for visual structure
- Enables actions to be applied to several selected cards or decks simultaneously
- Auto-closes when the user clicks elsewhere or selects an action

3.1.7 Replay

The *Replay* class manages the playback of previously recorded game sessions.

- Loads .csgf recording files and reads stored game actions
- Allows user to step forward or backward through actions
- Supports autoplay with adjustable speed
- Displays visual indicator when playback is complete
- Integrates with *Recording* to interpret saved actions

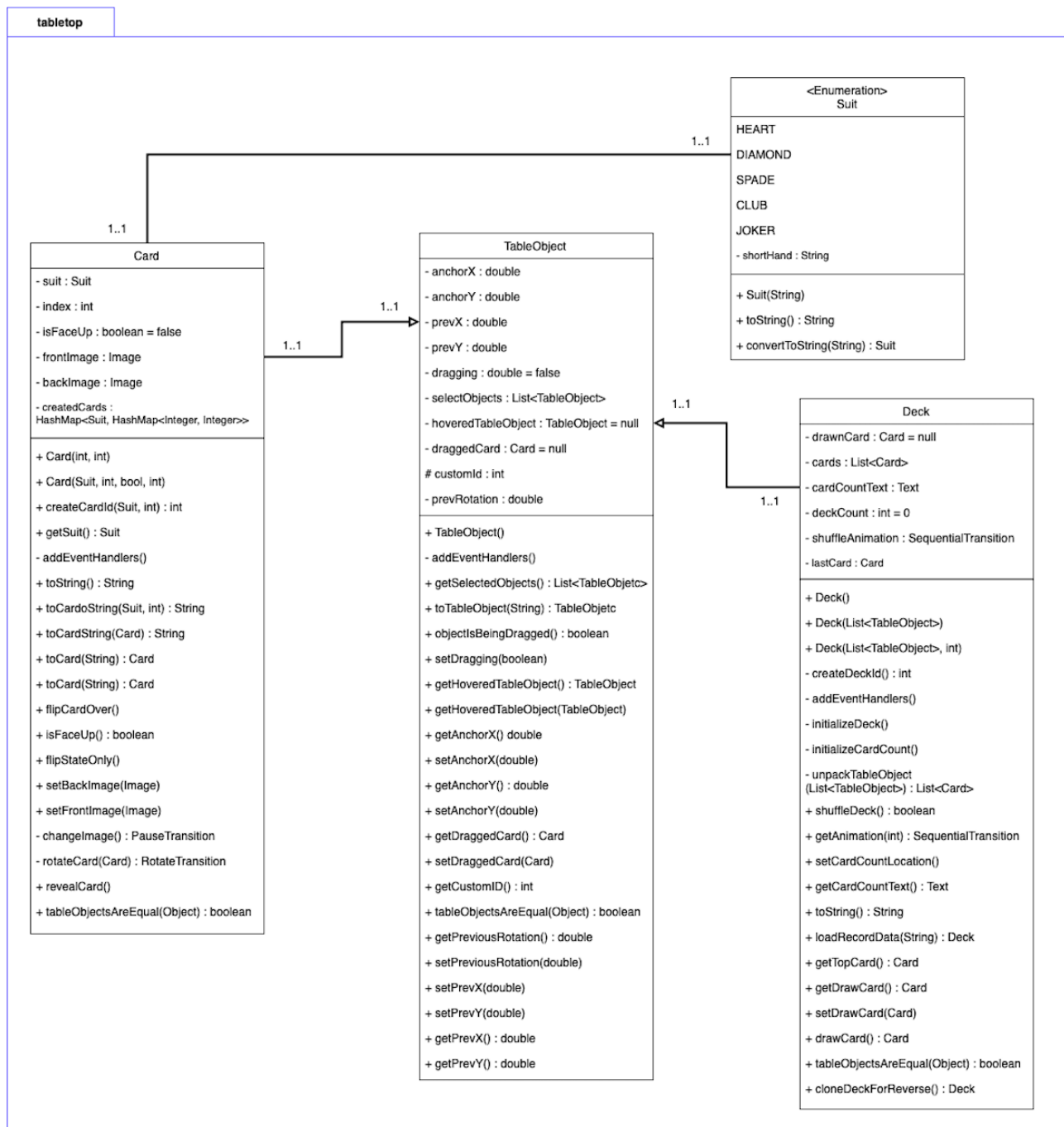


Figure 4: The Class Diagram for the tabletop Package

3.1.8 TableObject

The *TableObject* class acts as a base class for other objects on the game table. Its attributes are as follows:

- **anchorX** and **anchorY**: Table location reference for mouse movement
- **prevX** and **prevY**: Double that temporarily stores the previous X and Y locations respectively on the table of an object
- **Dragging**: boolean that records whether an object is currently being moved by the mouse
- **selectedObjects**: a list of table objects that are currently selected by control-clicking
- **hoveredTableObject**: a tableObject that the mouse is currently hovering over

- draggedCard a tableObject that is currently being dragged by the mouse
- customID: a protected integer
- prevRotation: a double for storing the previous assigned rotation of a card before manipulation

The *TableObject* Class has several methods to add functionality:

- A constructor to create a tableObject
- getSelectedObjects: returns the selectedObjects list of tableObjects
- toTableObject: creates a table object from a string of specific format
- objectsIsBeingDragged: returns the value of the variable dragging
- setDragging: setter for the variable dragging
- getHoveredTableObject: returns the table object stored in the variable hoveredTableObject
- setHoveredTableObject: sets the variable hoveredTableObject to the table object passed to this method
- Getter and Setters for the anchorX and anchorY variables
- Getter and setter for the draggedCard variable
- getCustomID: returns the integer value of the customID variable
- tableObjectsAreEqual: checks if two given tableObjects are equal returning a boolean based on the result
- Getter and setter for the prevRotation variable
- Getter and setter for the prevX and prevY variables

3.1.9 Suit

The enumeration of *Suit* provides the suits for the card class and some simple method for their use. These values include: HEART (H), DIAMOND (D), SPADE (S), CLUB (C) and JOKER (J) with their corresponding shortHand representation.

The methods provided in the *Suit* Enum are:

- The constructor
- toString: returns the string representation of shortHand
- convertToSuit: Takes a ShortHand representation of a suit and returns the corresponding Enum value

3.1.10 Card

The *Card* Class consists of several attributes:

- Suit: enum type that represents the card's suit (heart, diamonds, spades, clubs and joker)
- Index: integer value that represents the numerical value of the card as well as if it is a king, queen or ace
- isFaceUp: a boolean type that records if the card is face up (true) or face down (false)
- frontImage : the image to be displayed on the front of the card
- backImage the image to be displayed on the back of the card
- createdCards: hash map

The *Card* Class consists of several attributes:

- Two *Card* constructor methods used throughout the program
- `createCardID`: creates a unique ID for every card
- `getSuit`: returns the suit of the card
- `toString`: creates a string representation of a unique card
- `toCardString`: creates a two letter string representation of a unique card
- `toCard`: converts a string in specific format to a card object
- `flip`: flips the card over to reveal hidden face
- `isFaceUp`: returns the `isFaceUp` boolean value
- `flipStateOnly`: flips the card without affecting the UI
- `setBackImage`: sets the image on the back of the card to the one provided to the function
- `setFrontImage`: sets the image on the Front of the card to the one provided to the function
- `changeImage`: used when flipping the card to help the appearance of a smooth transition to the other face
- `rotateCard`: Creates a rotation animation to spin the card around the Y-axis for the flipping animation
- `revealCard`: flips the card for 5 seconds to reveal the hidden face
- `tableObjectsAreEqual`: checks if two given cards have the same index, suit and face up

3.1.11 Deck

This class represents a collection of *Cards*, offering methods for shuffling drawing and managing cards, its attributes are:

- `drawnCard`: a card type that stores the last card being drawn from the deck
- `Cards`: a list of card objects that are assigned to the deck
- `cardCountText`: text count of cards assigned to the deck
- `deckCount`: integer count of the decks currently created
- `shuffleAnimation`: a `sequentialTransition` used as an animation played during various operations on decks

The methods included in the *Deck* class are:

- Three constructors for creating decks at various points in the program
- `createDeckID`: returns the next integer ID for a newly created deck based on the `deckCount` variable
- `initializeDeck`: sequentially adds all required cards to a newly created deck
- `initializeCardCount`: initializes a card count to be displayed in the corner of the deck on the UI
- `unpackTableObject`: forms a deck of cards from a list of `tableObjects`
- `shuffleDeck`: shuffles the cards within a selected deck, returns a boolean `true` when completed
- `getAnimation`: creates an animation to be used when shuffling and moving cards within a deck, the number of times the animation loops is set by the integer passed to the method, the final animation is assigned to the `shuffleAnimation` variable

- Getter for the Cards list
- setCardCountLocation: positions the card count UI element on the deck
- Getter for the cardCountText
- toString: creates a string representation of a deck the deck and its unique ID and all the cards contained inside the deck in order
- loadRecordedDeckData: takes a string of specific format and creates a deck with accompanying cards, used for loading of recorded gameplay
- getTopCard: returns the card object at the top of the deck
- getDrawCard: returns the card object drawn from the deck
- setDrawnCard: sets the drawnCard variable to the given card object
- drawCard: moves the drawn card from the top of the deck to the table, if the deck has only one card once the top card is drawn the deck is replaced with the remaining card
- tableObjectsAreEqual: checks if two decks are equal, ignoring object identity and UI properties
- cloneDeckForReverse: creates a copy of the deck and removes the first card for use in replays

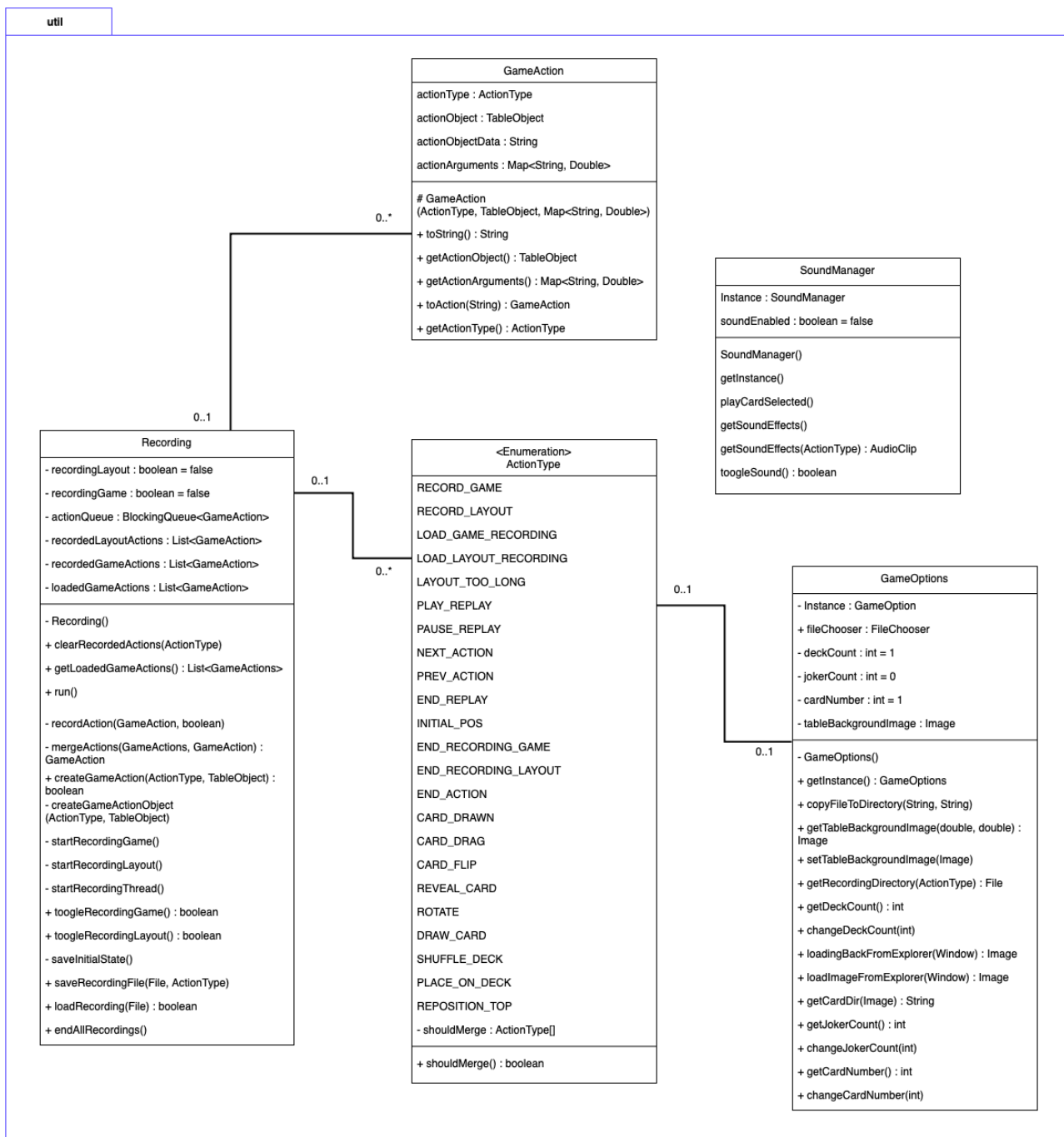


Figure 5: The Class Diagram for the util Package

3.1.12 GameOptions

The *GameOptions* class stores the user's game setup preferences and configuration settings.

- Stores configuration data such as number of decks, jokers, and custom image selections
- Provides getter and setter methods for values like deck count, joker count, and selected image paths
- Ensures customization settings stay within allowed limits, such as the maximum number of decks or jokers
- Includes toggle methods for features such as grid snapping and sound effects

3.1.13 ActionType

ActionType is an enumeration that defines all the possible types of actions that can occur during gameplay or UI interaction. These actions are used by the game engine to record, replay, and interpret player behavior and system events such as card movements, layout changes, or recording control signals.

- shouldMerge: determines whether multiple of an action type in a row should be merged into one action during recording

3.1.14 GameAction

The GameAction class models a single action taken during the game. It stores information about the type of action, the object involved (like a card or pile), and any relevant arguments (like coordinates). It's primarily used for saving, loading, and replaying game interactions.

Its functions are as follows:

- GameAction(ActionType actionType, TableObject actionObject, Map<String, Double> actionArgs): Protected constructor that initializes a new GameAction with its type, associated game object, and relevant action arguments.
- toString(): Returns a custom string representation of the action, combining its type, object data, and arguments—mainly used for saving or debugging.
- getActionObject(): Returns the TableObject (e.g., a card, pile, etc.) associated with this action.
- getActionArguments(): Returns a map of argument keys and their numeric values that define the specifics of the action (e.g., positions or sizes).
- toAction(String actionData): Static method that converts a string (typically from a saved file) back into a GameAction object, parsing the type, object, and arguments.
- getActionType(): Returns the ActionType enum value representing what kind of action this is (e.g., draw, move, place).

3.1.15 Recording

The *Recording* Class handles recording and loading replays and dealing layouts.

Its attributes include:

- recordingLayout: a boolean, true if a game recording is currently in progress and false if not
- recordingGame: a boolean, true if a layout recording is currently in progress and false if not
- actionQueue: a queue of gameActions to be recorded
- recordedLayoutActions: a list of recorded layout actions
- recordedGameActions: a list of recorded game actions
- loadedGameActions: a list of gameActions loaded from a previous recording file

The methods provided with the class are as follows:

- Recording: the Constructor for the class
- clearRecordedActions: clears the recordedLayoutActions and recordedGameActions lists at the end of a recording ready for a new recording to start
- Getter for the loadedGameActions list
- Run: controls the recording process and contains the main recording thread, the method continues running as long as any recording is in progress
- recordAction: records a single GameAction to the appropriate list

- `mergeActions`: combines two `GameActions` into one to simplify the recording process
- `createGameAction`: creates a `gameAction` from an `ActionType` and a `Tableobject`, multiple game actions can be created if an action requires
- `startRecordingGame`: starts the recording of a game
- `startRecordingLayout`: starts the recording of a dealing layout
- `startRecordingThread`: creates the thread for the recording process to run on
- `toggleRecordingGame`: toggles the game recording process on or off appropriately
- `toggleRecordingLayout`: toggles the layout recording process on or off appropriately
- `saveInitialState`: saves the initial state of all `tableObjects` and their locations
- `saveRecordingFile`: saves the recording file once a recording has been completed
- `loadRecording`: loads a previous recording from a specified file
- `endAllRecordings`: utilises the `toggleRecordingLayout` and `toggleRecordingGame` methods to end all running recordings

3.1.16 SoundManager

`SoundManager` is a utility class for handling sound effects in the game. Currently, it manages and plays a sound effect when a card is selected, using JavaFX's `AudioClip`. `cardSelectSound` is a private static `AudioClip` object that loads the audio file `ClickSoundTest.mp3` from the resources directory when the class is initialized. It is used to provide auditory feedback (e.g., when selecting a card). Its `playCardSelect()` plays a short audio clip (`ClickSoundTest.mp3`) to give audio feedback when a card is selected or interacted with. Useful for enhancing user experience with sound cues.

3.2 Discussion of Key Data Structures

This section is incorrect (as of 26th March, 2025) and will be updated soon. The content below is a mix of out of date and just incorrect.

The program uses a few key data structures to manage and organise data efficiently.

These structures have different functionalities within the system, such as player customization and Game recording data. Listed below are the key data structures used throughout the program:

3.2.1 Player Customization

- **Attributes:** This structure includes the decided card types and background the player(s) would like to use while playing the game.
- **Functionality:** This structure is important for the setup of game as it stays throughout and cannot be changed once decided.
- **Implementation:** This Structure is implemented as class and derived from the screen parent class where the setup is stored.

3.2.2 Game Recording Data

- **Attributes:** This structure included being able to record and replay previous games that have been played in the past.
- **Functionality:** This Structure is important for making the game repeatable and re playable as the User is able to watch previous setups and rules to follow along to.

- Implementation: This Structure is implemented as a class and derived from the Table parent class where a loading file is created, stored and ran upon request

3.3 Mapping from User Stories to Classes

Requirement	Classes Providing Requirement
US1 Number of Decks	Customization, GameOptions
US2 Number of Jokers per Deck	Customization, GameOptions
US3 Card Images	Customization, CardFrontCustomization, GameOptions
US4 Table Background Image	Customization, Table, GameOptions
US5 Shuffling Decks	Deck, Popup
US6 Move a Card	TableObject, Table
US7 Reposition a Card in a Group	Card, Deck, TableObject, Popup
US8 Move Multiple Cards	TableObject, Table
US9 Flip a Card	Card, Popup
US10 Flip Multiple Cards	Card, Popup
US11 Reveal Cards	Card, Popup
US12 Save Dealing Layout	Card, Deck, Table, Recording, GameAction
US13 Deal Cards From Layout	Table, Card, Deck, Replay, Recording, GameAction
US14 Rotate Cards	TableObject
US15 Record Games	Card, Deck, Table, Recording, GameAction
US16 Replay Games	Card, Deck, Table, Replay, Recording, GameAction
US17 Snap to Grid	TableObject, Table
US18 Sound Effects	SoundManager

4 DYNAMIC DESCRIPTION

4.1 Sequence Diagrams and Pseudocode

US1: Number of Decks

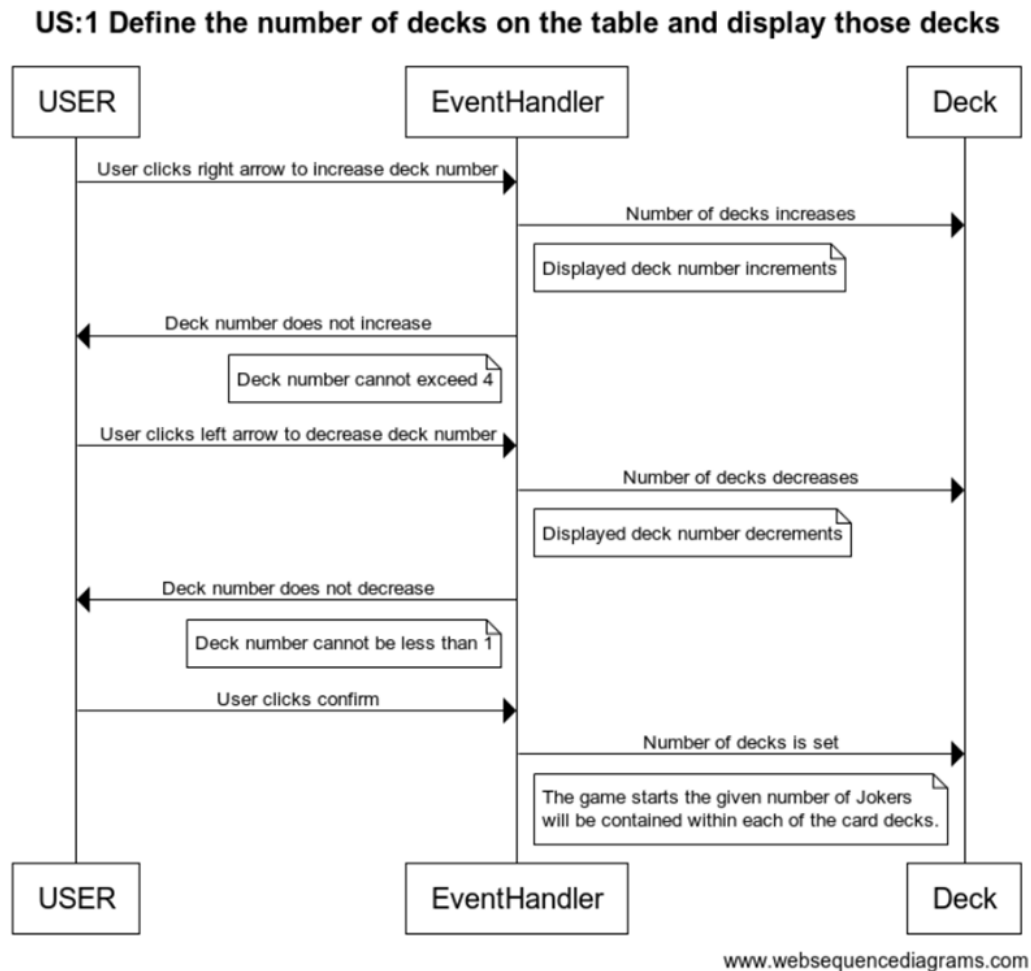


Figure 6: US1 Sequence Diagram

This sequence diagram outlines how a user sets the number of card decks for a game. The user can increase or decrease the deck count by clicking the right or left arrows, respectively.

The event handler for click events updates and displays the new count but restricts the number to a minimum of 1 and a maximum of 4. Once the user clicks confirm, the event handler causes Deck to initialise the appropriate number of decks which are to be displayed in the game.

```
1 CLASS GameOptions
2     INTEGER DeckCount <- 0
3
4     METHOD GetDeckCount()
5         RETURN DeckCount
6     END METHOD
7
8     METHOD ChangeDeckCount(Count)
9         IF (DeckCount + Count) >= 1 AND (DeckCount + Count) <= 4 THEN
10             DeckCount <- DeckCount + Count
11         END IF
12     END METHOD
13 END CLASS
14
15 CLASS Customization
16     OBJECT OptionsInstance <- NEW GameOptions
17     OBJECT IncreaseDeckCountButton <- NEW Button LABELED "+"
18     OBJECT DecreaseDeckCountButton <- NEW Button LABELED "-"
19
20     SET ACTION IncreaseDeckCountButton ON CLICK
21         CALL OptionsInstance.ChangeDeckCount(1)
22     END ACTION
23
24     SET ACTION DecreaseDeckCountButton ON CLICK
25         CALL OptionsInstance.ChangeDeckCount(-1)
26     END ACTION
27 END CLASS
```

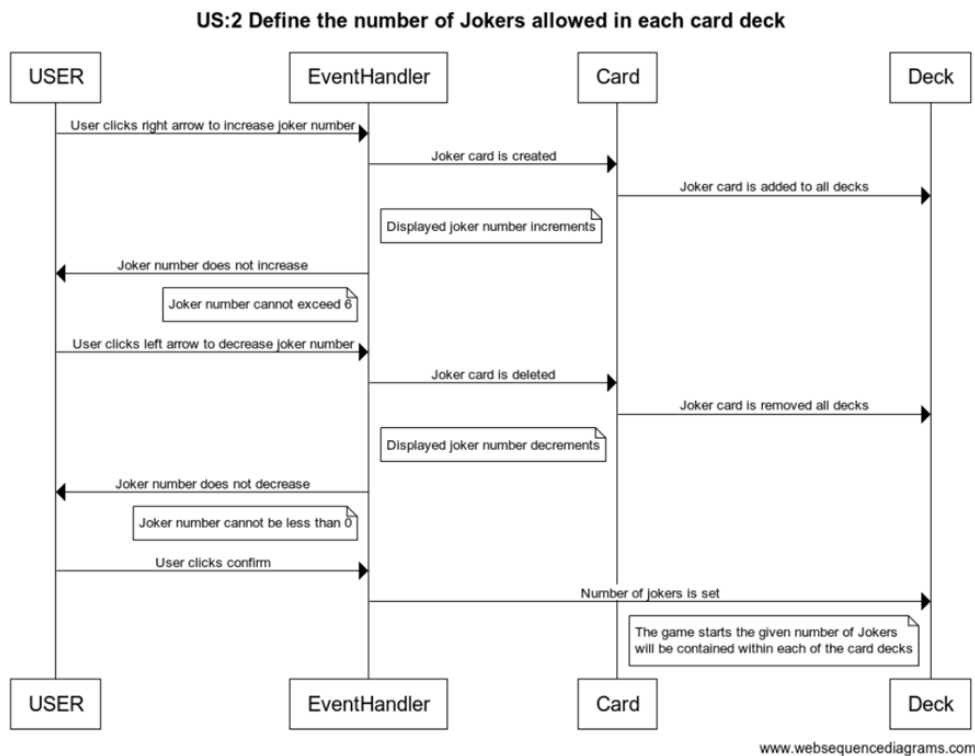
US2: Number of Jokers per Deck

Figure 7: US2 Sequence Diagram

This sequence diagram details the process for configuring the number of Jokers to be included in each card deck before the game begins. The user can increase or decrease the deck count by clicking the right or left arrows, respectively. When increasing, a Joker Card is created and added to all Deck's, while decreasing results in a Joker Card being removed. The system enforces limits, allowing a minimum of 0 and a maximum of 6 Jokers per Deck. Upon confirmation, the selected number of Jokers is set, and the game starts with the specified configuration applied to each Deck.

```

1  // GameOptions Class
2  CLASS GameOptions
3      PRIVATE INTEGER JokerCount = 0
4
5      PUBLIC METHOD GetJokerCount()
6          RETURN JokerCount
7      ENDMETHOD
8
9      PUBLIC METHOD ChangeJokerCount(INTEGER Count)
10         IF Count IS within acceptable joker range THEN
11             JokerCount = JokerCount + Count
12         ENDIF
13     ENDMETHOD
14 ENDCLASS
15

```

```

16 // Customization Class
17 CLASS Customization
18     Button increaseJokerCountButton = NEW Button("+")
19     Button decreaseJokerCountButton = NEW Button("-")
20
21     increaseJokerCountButton.SETONACTION(event -> GameOptions.
        ChangeJokerCount(1))
22     decreaseJokerCountButton.SETONACTION(event -> GameOptions.
        ChangeJokerCount(-1))
23 ENDCLASS

```

US3: Card Images

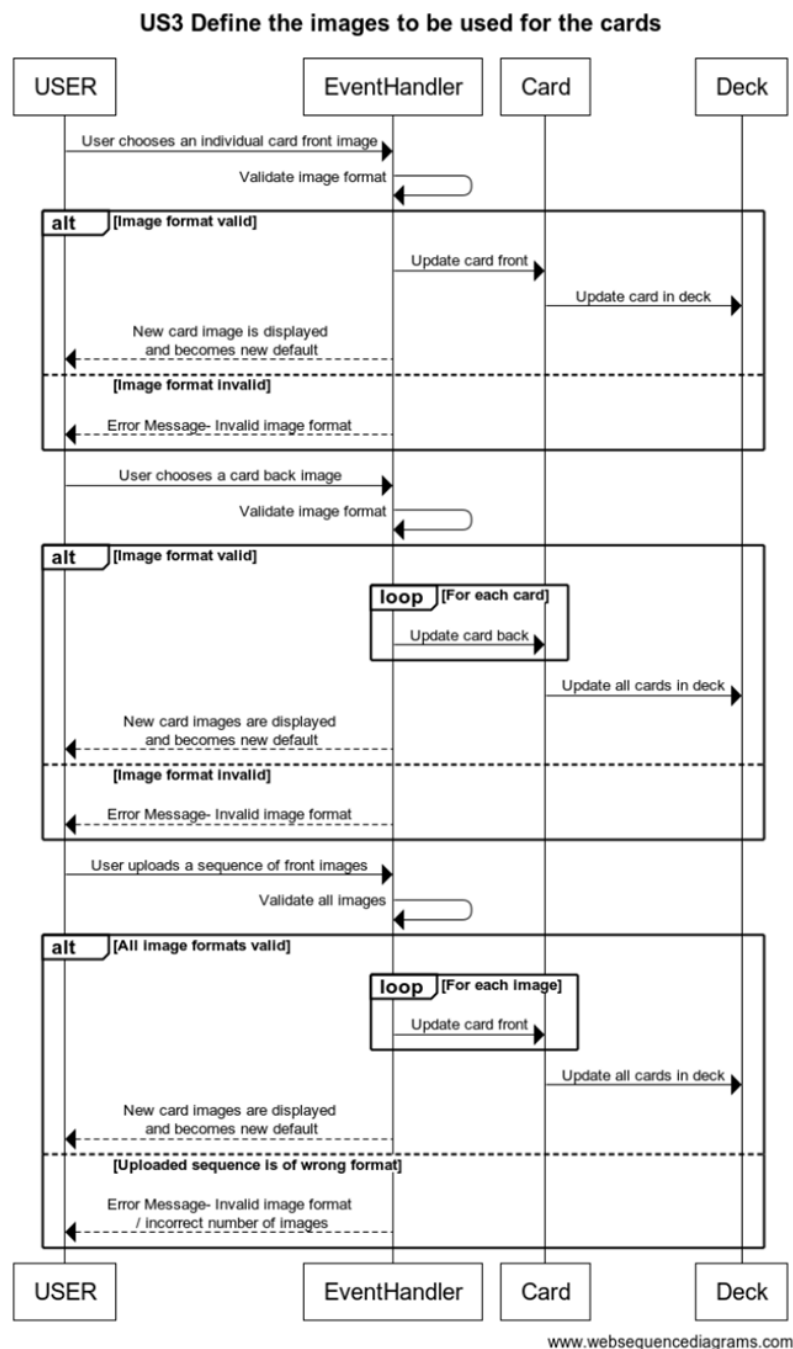


Figure 8: US3 Sequence Diagram

This sequence diagram describes the user interaction for customising card images used in the game. The user can select an individual front image, a back image, or upload a sequence of front images. For each action, the system first validates the image format. If valid, the selected image(s) are applied to the respective Cards, and the updated visuals are reflected in the Deck, becoming the new default. For individual images, the change is applied to the Card or all Cards as appropriate. When the user uploads a sequence of images, each Card front is updated accordingly. If any image format is invalid or the sequence is incorrect, the event handler that got the images displays an appropriate error message, preventing the update.

```
1 BEGIN selectCardImage
2   CHECK image has not been selected
3   CREATE a fileChooser title to "tap to change card image"
4
5   DEFINE fileFilters for images (JPG, PGN, etc...)
6   APPLY fileFilters to FileChooser
7
8   OPEN fileChooser dialog and STORE selected files
9   IF file is NOT selected THEN
10     EXIT function
11   ENDIF
12
13   CONVERT selected file to an Image object
14
15   CREATE a CardImage using image
16   CREATE a Card with the CardImage
17
18   APPLY FrontCard to ALL Cards
19
20   IF singleCard is selected to have a new CardImage
21     FOR singleCard:
22       SELECT new image
23       APPLY new image to single Card
24     UPDATE deck
25   ENDIF
26 END selectCardImage
```

US4: Customize Background

US4 Define image used for the table background

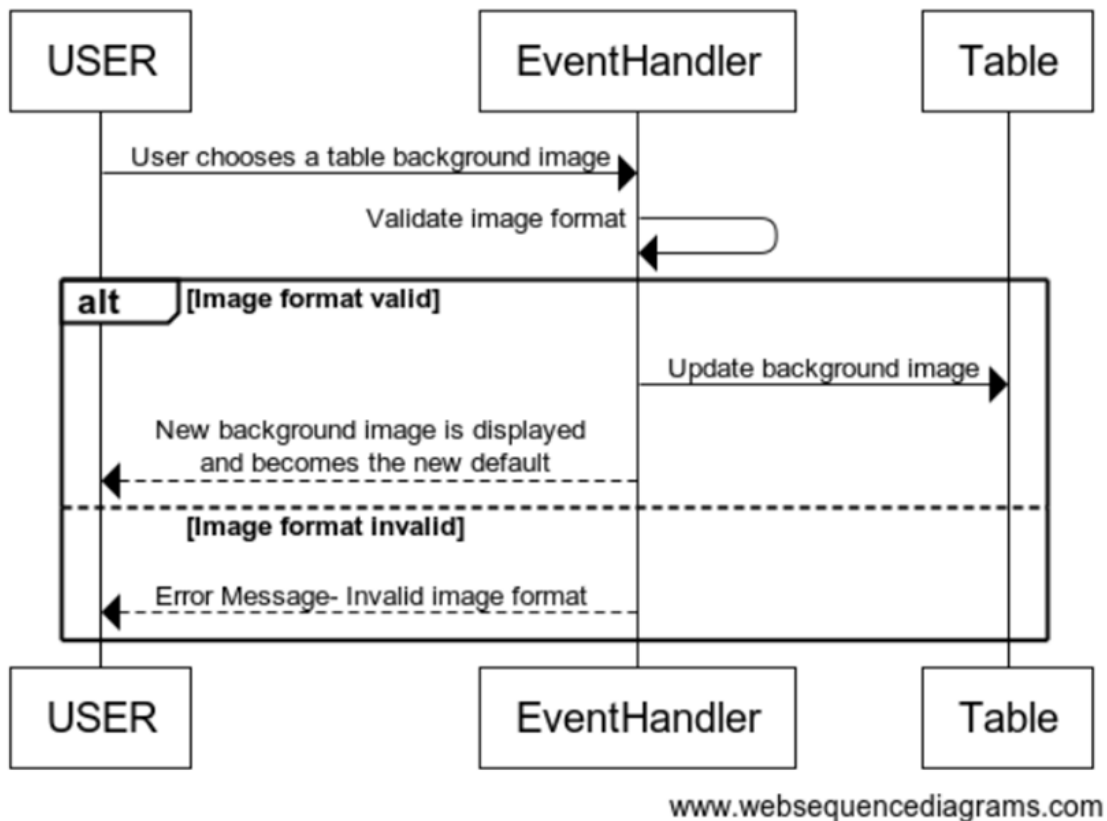


Figure 9: US5 Sequence Diagram

This sequence diagram outlines the process for setting a custom table background image. When the user selects an image, the event handler that was used to select it validates its format. If the image is valid, the background is updated on the table, and the new image is displayed as the default. If the image format is invalid, the event handler notifies the user with an appropriate error message, and the background remains unchanged.

```

1 BEGIN SelectAndSetBackgroundImage
2   CREATE a FileChooser object
3   SET fileChooser title to "Select an Image"
4
5   DEFINE fileFilters for images (JPG, PNG, etc.)
6   APPLY fileFilters to fileChooser
7
8   OPEN fileChooser dialog and STORE selected file
9   IF file is NOT selected THEN
10     EXIT function
11   ENDIF
12
13   CONVERT selected file to an Image object
14   CREATE a BackgroundImage using the Image
15   CREATE a Background with the BackgroundImage
  
```



```

16
17     APPLY Background to the thumbnail/board background
18 END SelectAndSetBackgroundImage

```

US5: Shuffling Decks

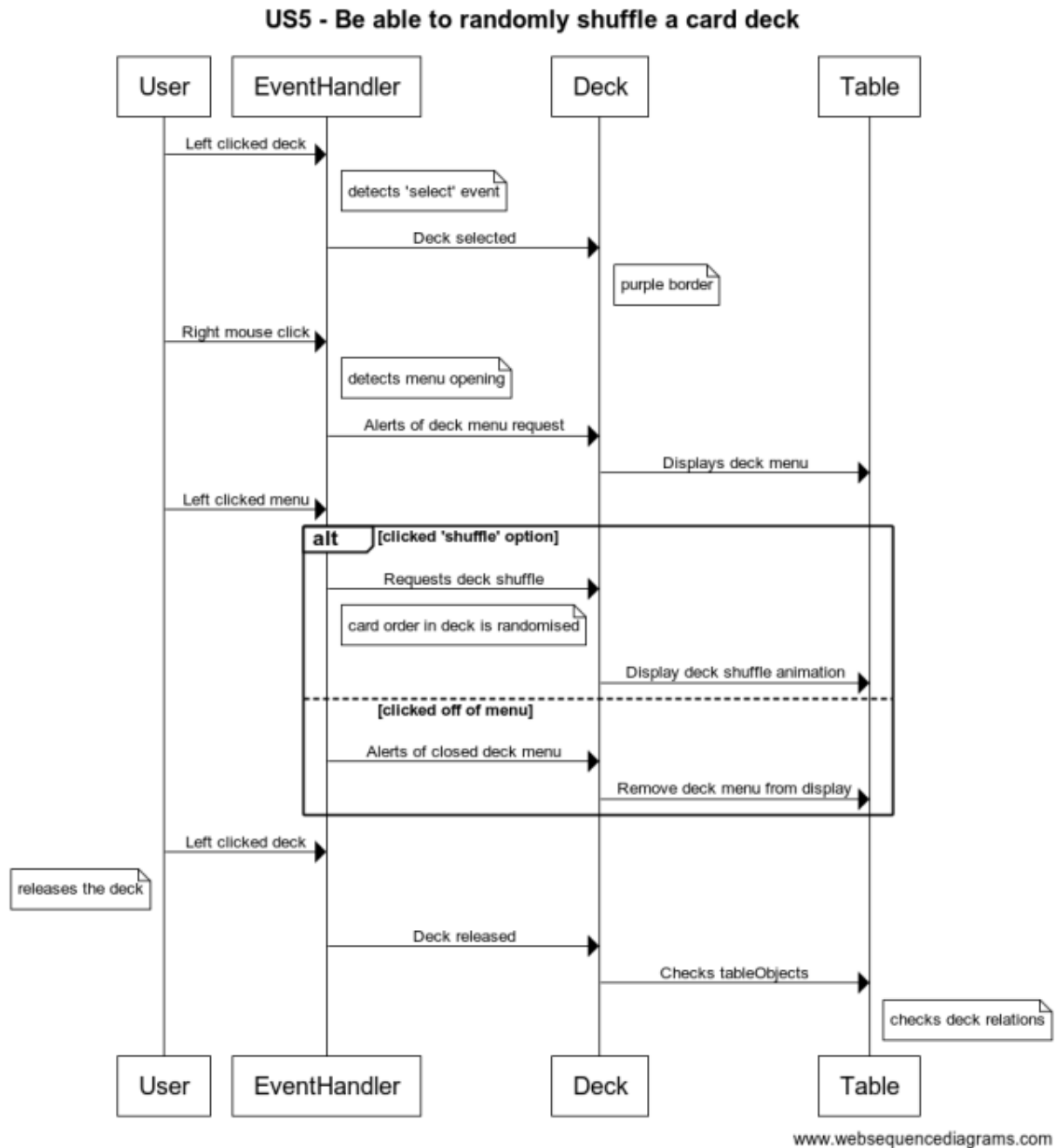


Figure 10: US5 Sequence Diagram

The sequence diagram above shows how a user randomly shuffles a selected card deck. When the user clicks on the deck, the system detects this as a selection event, highlighting the deck with a purple border to indicate its new state. A right-click triggers a menu-opening event, resulting in the deck menu being displayed. If the user selects the shuffle option, the system randomises the order of the cards within the deck and triggers a shuffle animation. If the user instead decides to click away from the menu, the system closes the menu, removing it from view. Finally, when the user releases the deck, the system marks the deck as released, and the Table component checks for any relational updates.

between the deck and other table objects.

```

1 Shuffle(deck)
2   IF deck IS OF TYPE Deck THEN
3     Randomly shuffle the cards in deck
4     Update the deck with the shuffled order
5     Play shuffle animation
6   END IF
7 END

```

US6: Move a Card

US6 - Use the mouse/trackpad input to move a single card around the table

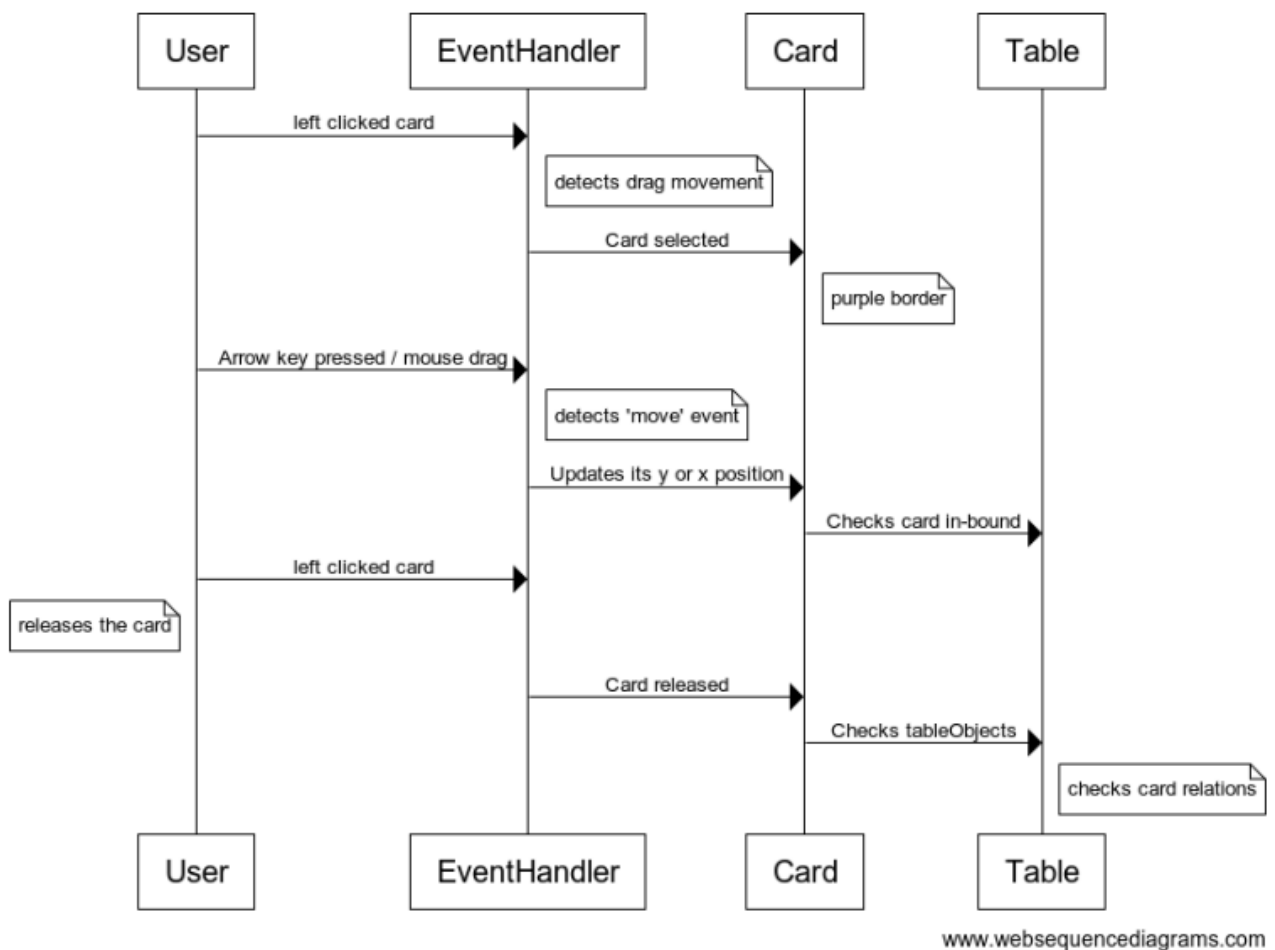


Figure 11: US6 Sequence Diagram

The sequence diagram above shows how a user moves a single card across the table using either arrow keys or drag-and-drop input. When the system detects the user's initial card click, the selected card is visually highlighted with a purple border. As the user moves the card using input controls, the system registers this as a movement event and updates the card's x or y position accordingly. The card then checks whether it remains within the table bounds, and once the user releases the card, the system finalises the action and checks for any interactions between the card and other table objects.

```

1 selectedCard = NULL
2 offsetX = 0
3 offsetY = 0
4

```

```
5 // Event handler for mouse press (when the user clicks on a card)
6 ON MousePressed(Event):
7     IF mouse click is on a card:
8         selectedCard = card clicked
9
10        // Record the position of the mouse relative to the card's
11        // position
12        offsetX = mouse X position - selectedCards X position
13        offsetY = mouse Y position - selectedCards Y position
14    END IF
15END MousePressed
16
17// Event handler for mouse drag (when the user drags the card)
18ON MouseDragged(Event):
19    IF selectedCard is NOT NULL:
20        // Update the position of the selected card to follow the
21        // mouse
22        newX = mouse X position - offsetX
23        newY = mouse Y position - offsetY
24
25        // Move the selected card to the new position
26        selectedCards X position = newX
27        selectedCards Y position = newY
28    END IF
29END MouseDragged
30
31// Event handler for mouse release (when the user releases the card)
32ON MouseReleased(Event):
33    // Once the card is released, stop tracking the card
34    selectedCard = NULL
35END MouseReleased
```

1. Card Selection: The user selects a card using a mouse or keyboard shortcut. The selected card's current position and orientation (face-up or face-down) are stored.
2. Card Movement: The user drags the card with a mouse or uses keyboard input (e.g., arrow keys) to move the card. As the user moves the card, its position is updated dynamically. The position is checked to ensure that it stays within the bounds of the table (no off-screen movement).
3. Card Placement: When the user stops dragging or releases the mouse (or presses a key), the card is finalized in its new position. If the new position overlaps with another card, the system resolves the overlap by stacking the cards according to their layering rules (e.g., top card or underneath). The card's orientation is maintained (it stays face-up or face-down as it was when picked up). The card is displayed in the new position.

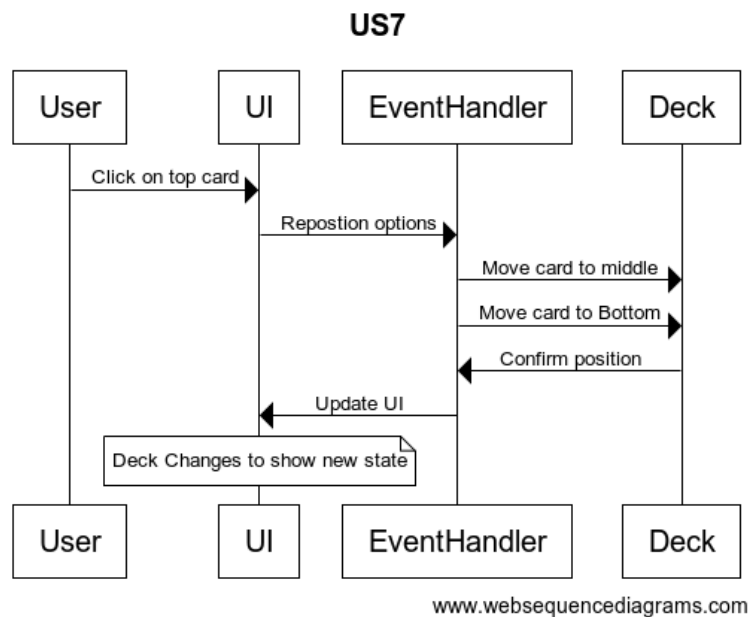
US7: Reposition a Card in a Group

Figure 12: US7 Sequence Diagram

In this interaction, the system allows a user to reposition a card within a group of cards to ensure smooth and flexible gameplay.

The user can select the top card from a group and move it to a different position within that same group.

The card can be placed at the bottom or somewhere else in the group through a popup option interaction activated by the right click of the mouse.

When the user selects the card, the UI communicates with the group using the click event handler, which handles the repositioning and updates the group accordingly.

The UI provides visual feedback while the repositioning is happening and again when it is complete, confirming the card has been moved successfully.

```

1 // Function To Reposition The Top Card Within A Selected Deck
2 FUNCTION
3     // Remove topCard from selectedDeck so that it can then be moved
4     // to its updated position
5     DEFINE topCard <- selectedDeck[0]
6
7     IF moveToBottom IS TRUE THEN
8         APPEND topCard TO top OF selectedDeck
9     ELSE
10        // Randomly select a new index position for the topCard
11        // somewhere in the deck
12        randomIndex <- RANDOM GENERATED INTEGER FROM 1 TO length of
13        // selectedDeck
14        INSERT topCard AT randomIndex IN selectedDeck
15    ENDIF
16
17    RETURN selectedDeck
18 END FUNCTION
  
```

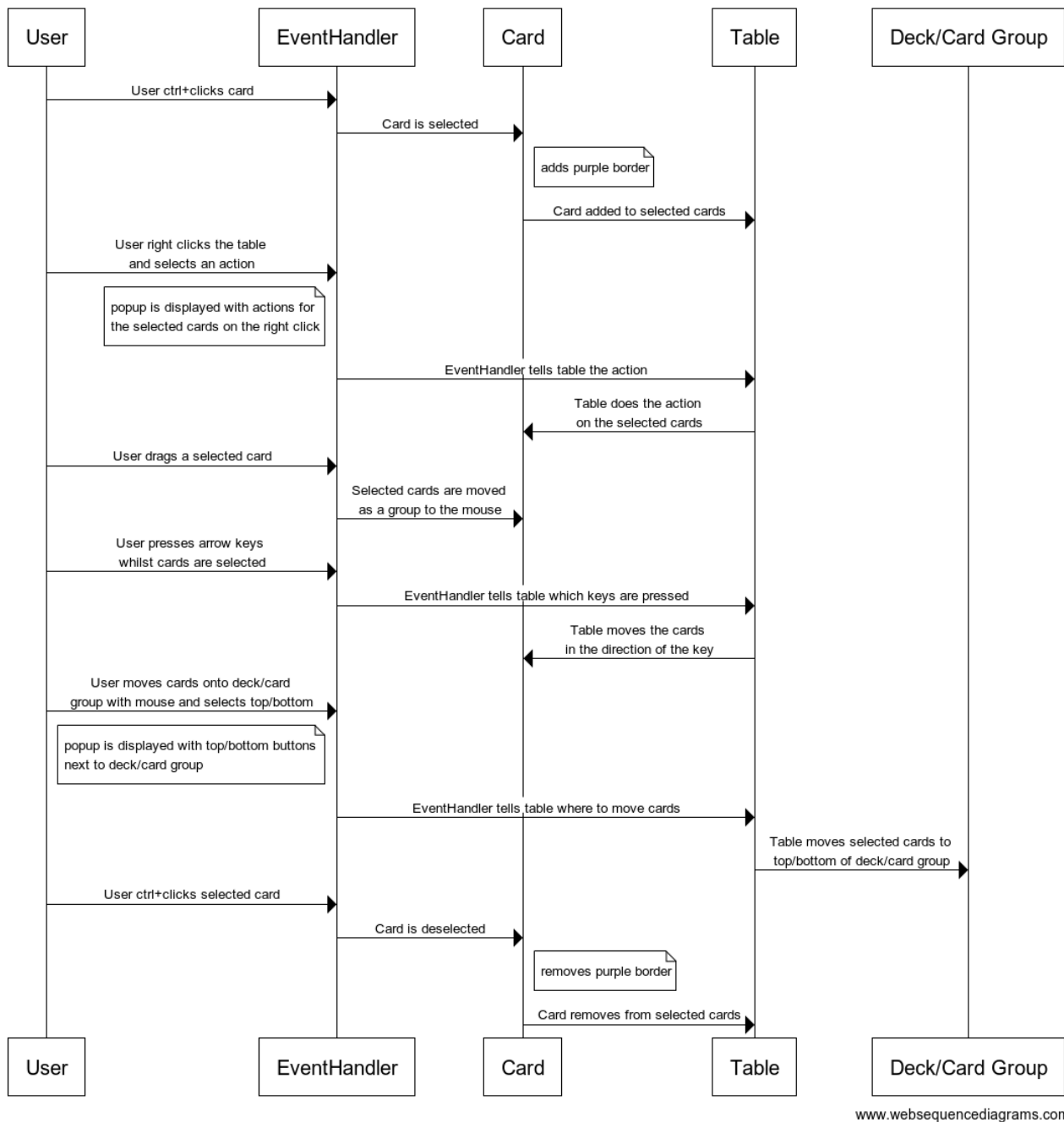
US8:Move Multiple Cards**US8 Use the mouse or trackpad or keyboard shortcut input to move a player's cards around the table**

Figure 13: US8 Sequence Diagram

Step 1: User Selects a Subset of Cards

WAIT for input: The program waits for the user to select cards, either through mouse clicks or keyboard shortcuts.

IF cards selected **THEN**:

Once the user selects cards, the program:

- **SELECT** a subset of cards (this could involve clicking or dragging multiple cards).
- **REVEAL** faces of selected cards (if the cards are face-down, they are flipped to face-up for easier decision-making).
- **STORE** selected cards in a list - The program keeps track of which cards are selected for further manipulation.

Step 2: User Moves the Selected Cards

WAIT for drag or keyboard input: The program waits for the user to start moving the cards, either by dragging them with the mouse or using keyboard keys.

IF dragging or keyboard movement starts THEN:

- DISPLAY visual feedback: The program highlights or outlines the cards being moved to give feedback to the user.
- STORE initial position: The starting position and orientation of the cards are saved to keep track of where they were originally placed.

WHILE dragging or moving with keyboard:

As long as the user continues to drag or use keyboard input:

For each card:

- If the user presses arrow keys or WASD, the cards are moved in the corresponding direction.
- If the user is dragging with the mouse, the cards' positions are updated based on the mouse's movement.
- Ensure cards stay on the table: The program checks that cards do not move off the visible table area. If they do, their position is adjusted to stay within bounds.
- Display updated position: The visual feedback is updated to show the new location of the cards.

Step 3: User Drops the Cards

WAIT for mouse release or keyboard stop:

The program waits until the user releases the mouse or stops using keyboard input.

IF release THEN:

The program checks where the cards were dropped:

- On a deck: If the cards are dropped on a deck, they are placed on top of it.
- In an empty space: If the cards are dropped in an empty space, they are placed there.

Ensure cards' orientation remains intact: After being dropped, the cards' orientations (face-up or face-down) are restored to their original state.

Display final feedback: The program provides visual feedback that the operation is complete, such as fading out highlights or animating the cards being placed.

Step 4: Reset UI

Remove feedback: The program removes any visual feedback (like highlights or outlines) used during the move.

Reset the UI: It resets the user interface, including deselecting any selected cards - Update card positions: The final positions of the cards are updated and stacked appropriately.

```
1 START
2 // Step 1: User selects a subset of cards
3 WAIT for input to select cards
4 IF cards selected THEN
5     SELECT subset of cards
6     REVEAL faces of selected cards
7     STORE selected cards in a list
8 END IF
9
10 // Step 2: User moves the selected cards
11 WAIT for drag or keyboard input
12 IF user starts dragging or moves with keyboard THEN
13     DISPLAY visual feedback
14     STORE initial position and orientation
15
16     WHILE dragging or moving with keyboard DO
17         FOR each card DO
18             IF arrow/WASD keys THEN MOVE card in direction
19             ELSE IF mouse drag THEN UPDATE position with offset
20         END FOR
21
22         FOR each card DO
23             IF position off table THEN ADJUST position
24         END FOR
25
26         DISPLAY updated position
27     END WHILE
28 END IF
29
30 // Step 3: User drops cards
31 WAIT for mouse release or keyboard stop
32 IF release THEN
33     IF dropped on deck THEN PLACE on deck
34     ELSE IF dropped in empty space THEN PLACE in space
35
36     FOR each card DO
37         SET card orientation to original
38     END FOR
39
40     DISPLAY final feedback
41 END IF
42
43 // Step 4: Reset UI
44 REMOVE feedback
45 RESET UI (deselect cards, reset highlights)
46 UPDATE card positions
47 END
```

US9: Flip a Card

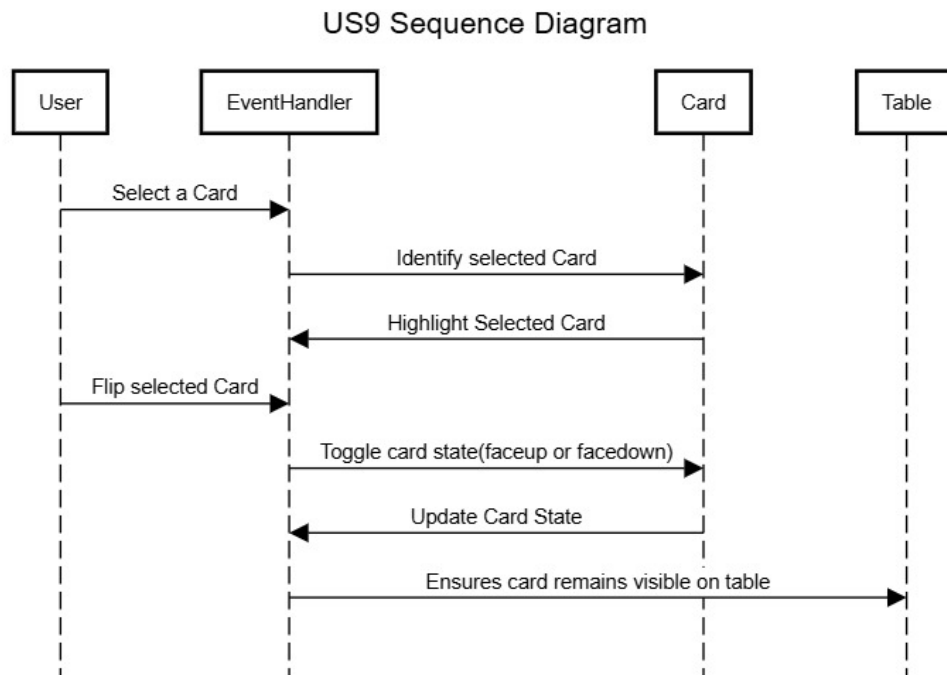


Figure 14: US9 Sequence Diagram

The sequence diagram above shows how a user selects and flips a card during a game. The event handler for the flip button processes the selection, identifies the card by hovering over it, and highlights it. When the user flips the card, the flip button event handler updates its state via the Card's class. A flipped card is then displayed on the table.

US10: Flip Multiple Cards

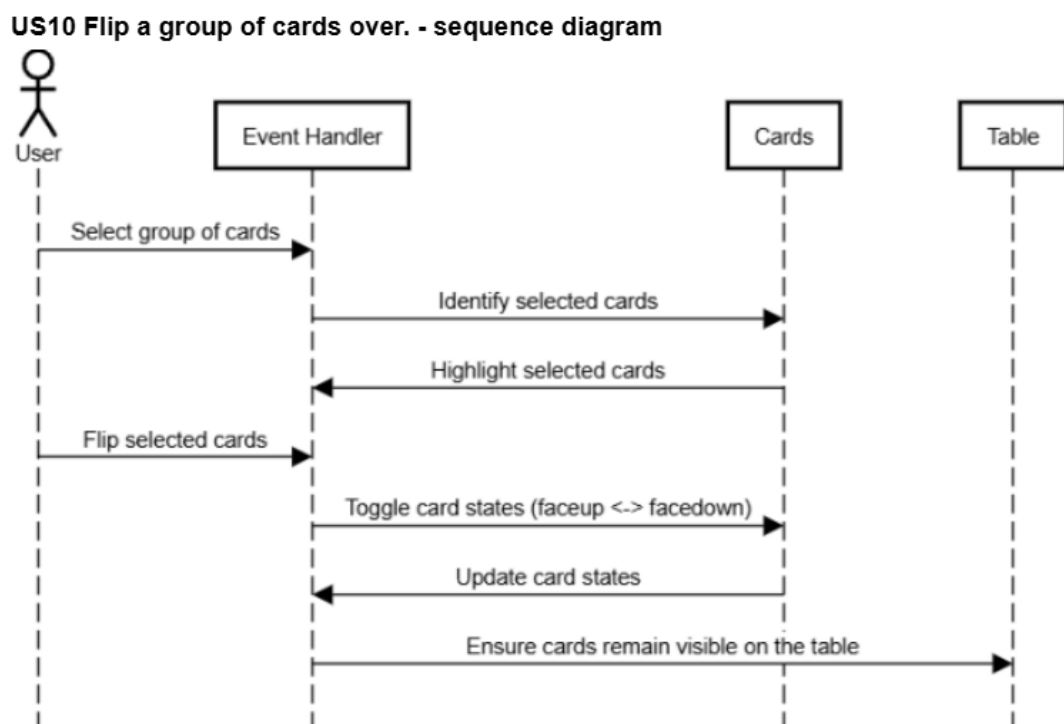


Figure 15: US10 Sequence Diagram

The sequence diagram above shows how a user selects and flips cards during a game. The event handler for the flip button processes the selection, identifies the cards by hovering over them, and highlights them. When the user flips the cards, the flip button event handler updates their states via the Card's class. The flipped cards are then displayed on the table.

US11: Reveal Cards

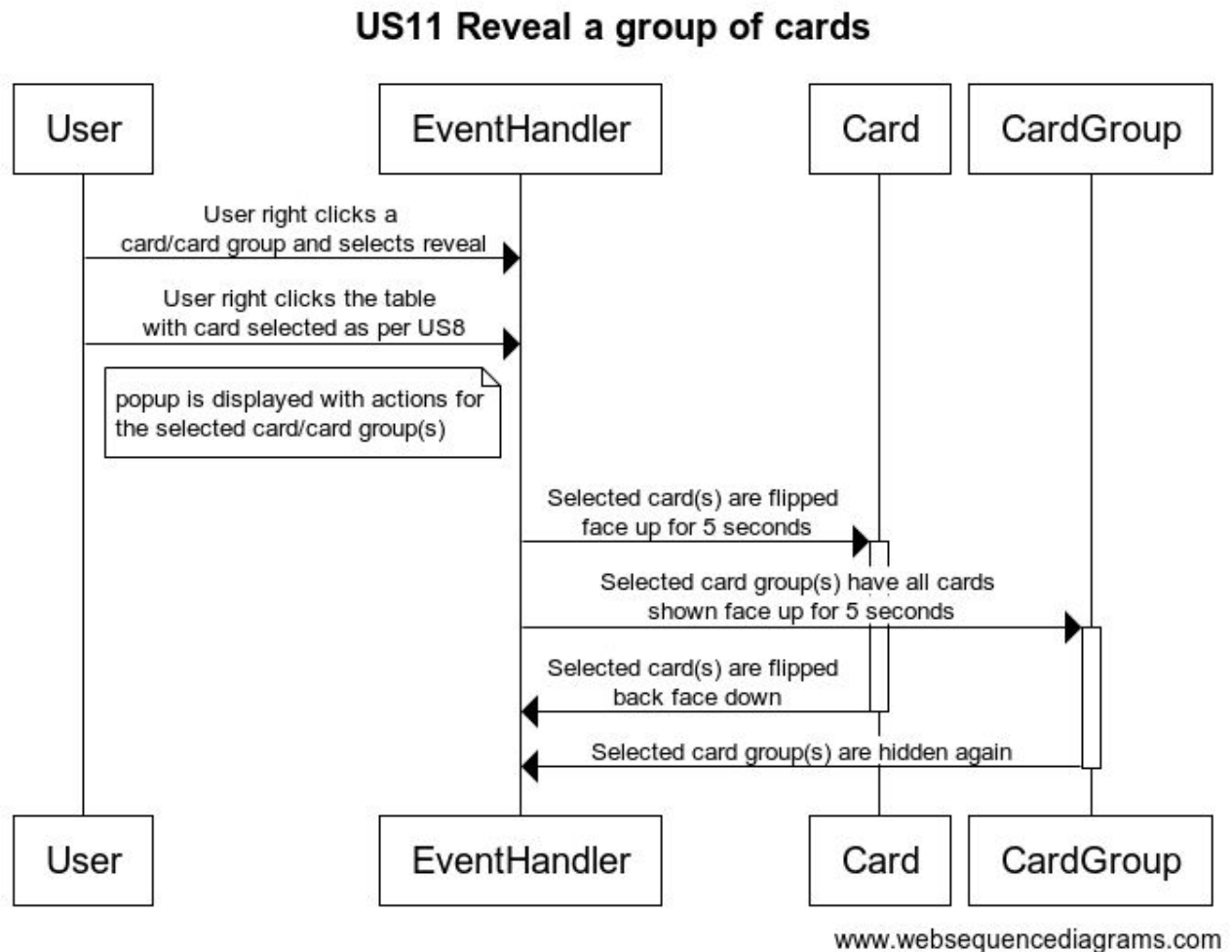


Figure 16: US11 Sequence Diagram

The US11 Sequence Diagram outlines the process for revealing a Card or Card group through the user interface. The user initiates this by right clicking a Card or group and selecting "reveal" from the popup menu, triggering the button's event handler to flip the selected Card(s) face up for 5 seconds before flipping them back down. If a group of cards is selected, all Cards within it are revealed simultaneously. The Card entities handle the visual state changes.

Pseudocode for US9, US10, US11

```
1 // have a data structure which keeps track of what objects have been
  selected
2 DEFINE selected = cards
3
4 // if left click action detected, check if clicked object can be added
  to selected list
5 IF action == leftClick THEN
6   // if yes then append to data structure
7   IF selected.type == selectedObj THEN
8     selected[+1] = selectedObj
9   ENDIF
10
11 // if right click action detected, depending on the type of objects
  selected, either open single card or group of cards menu
12 ELSEIF action == rightClick THEN
13   IF selected.type == single THEN
14     // depending on button clicked, act appropriately on the
      selected case
15     SWITCH singleMenu
16       CASE flip:
17         // for loop goes through all selected cards and
          reverses 'cardFacing' bool
18         selected.flip()
19       CASE reveal:
20         selected.flip()
21         delay(5s)
22         selected.flip()
23       CASE rotate:
24         // Rotate
25       CASE default:
26         leaveMenu
27     ENDSWITCH
28   ELSE THEN
29     // depending on button clicked, act appropriately on the
      selected case
30     SWITCH groupMenu
31       CASE flip:
32         selected.flip()
33       CASE shuffle:
34         // Shuffle
35       CASE reveal:
36         selected.flipAll()
37         delay(5s)
38         selected.flipAll()
39       CASE rotate:
40         // Rotate
41       CASE default:
42         leaveMenu
43     ENDSWITCH
44   ENDIF
45 ENDIF
```

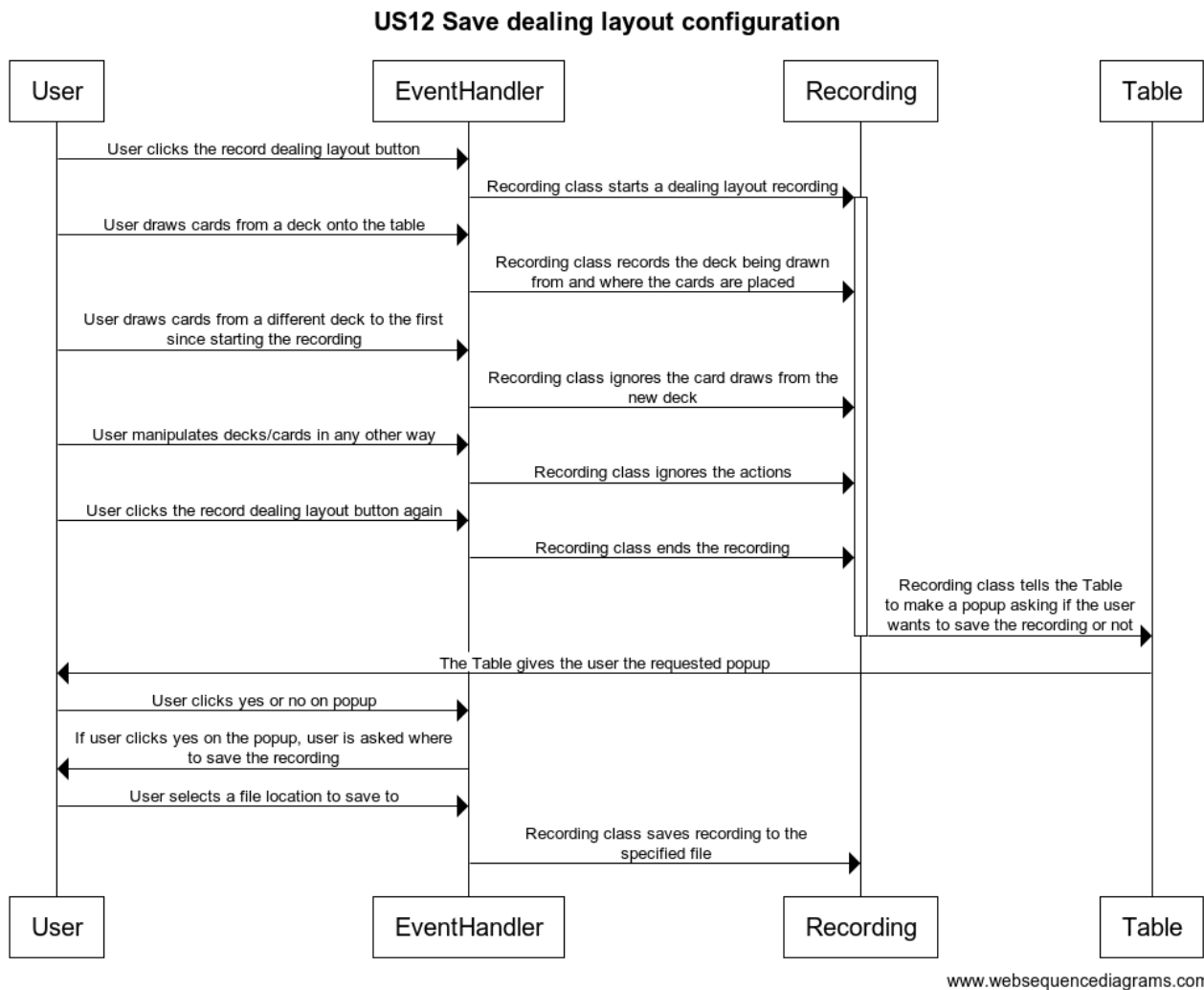
US12: Save Dealing Layout

Figure 17: US12 Sequence Diagram

The sequence diagram above shows how a user records and saves a dealing layout configuration. When the user clicks the record layout button, the system begins tracking card placements from the selected deck. As the user draws cards from the selected deck onto the table, the recording class captures each placement. If the user performs unrelated actions, they are ignored by the system. When the user clicks the record button again, the recording stops and a popup appears asking whether to save the layout. If the user confirms, they are prompted to choose a save location, and the recording is saved.

```
1 // When dealing layout recording button is pressed
2 createNewCardActionRecordingThread()
3
4 // After any action is taken in game
5 GameAction gameAction = createGameActionFromEvent()
6 actionQueue.add(gameAction)
7
8 // To end a dealing layout recording (same button as starting one)
9 GameAction endAction = createGameActionDealingEndSignal(shouldSave)
10 actionQueue.add(endAction)
11
12 // In the recording thread
13 recording = true
14 WHILE (recording)
15     // thread safe, waits for update
16     action = getActionQueue().take()
17
18     // If it should end, end
19     IF (isEndSignal(action))
20         IF (action.shouldSave())
21             saveRecordingQueueToFile()
22         ENDIF
23         recording = false
24     ENDIF
25
26     // If the action is something that works incrementally like
27     // rotating, it merges it with the
28     // previous action so you dont have an action for each degree of
29     // rotation of the same card
30     action = checkMergeAction(action)
31
32     // If only recording a dealing layout, ignore irrelevant actions
33     IF (!isCardAction(action) AND !isRecordingAll)
34         continue;
35     ENDIF
36
37     // Push it to a queue to be saved
38     recordingQueue.add(action)
39 NEXT
```

US13: Deal Cards From Layout

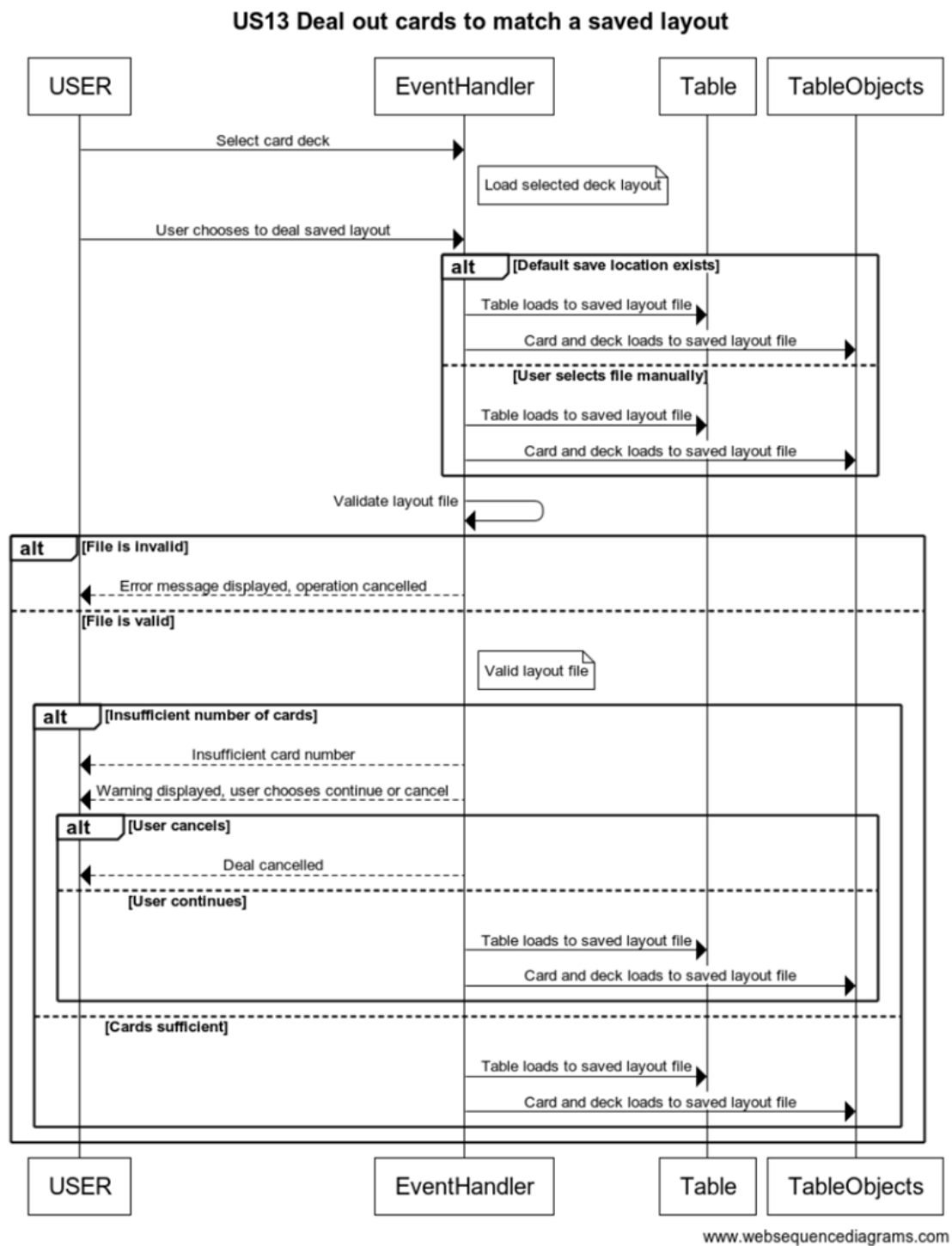


Figure 18: US13 Sequence Diagram

This sequence diagram describes the process of dealing cards to match a previously saved table layout. The user first selects a card deck, triggering the deck's event handler to load the corresponding layout. When the user chooses to deal the saved layout, the system attempts to load the layout either from a default location or a user-selected file. The layout file is then validated. If the file is invalid, an error message is shown, and the operation is cancelled. If valid, the system checks whether there are enough cards to complete the layout. If not, the user is warned and given the option to proceed or cancel. If

the user proceeds, or if the card count is sufficient from the start, the table and associated cards are dealt based on the saved configuration.

```

1 deck=card_deck
2
3 If the file chosen exists then get dealing layout file
4 else display an error
5
6 If the number of instructions for dealing is more then the size of
  deck chosen then issue a warning
7
8 Once conditions are checked and file is validated,
9
10     Until instructions are available, parse the file instructions and
      perform the action
11
12     Print message specifying completion
  
```

US14: Rotate Cards

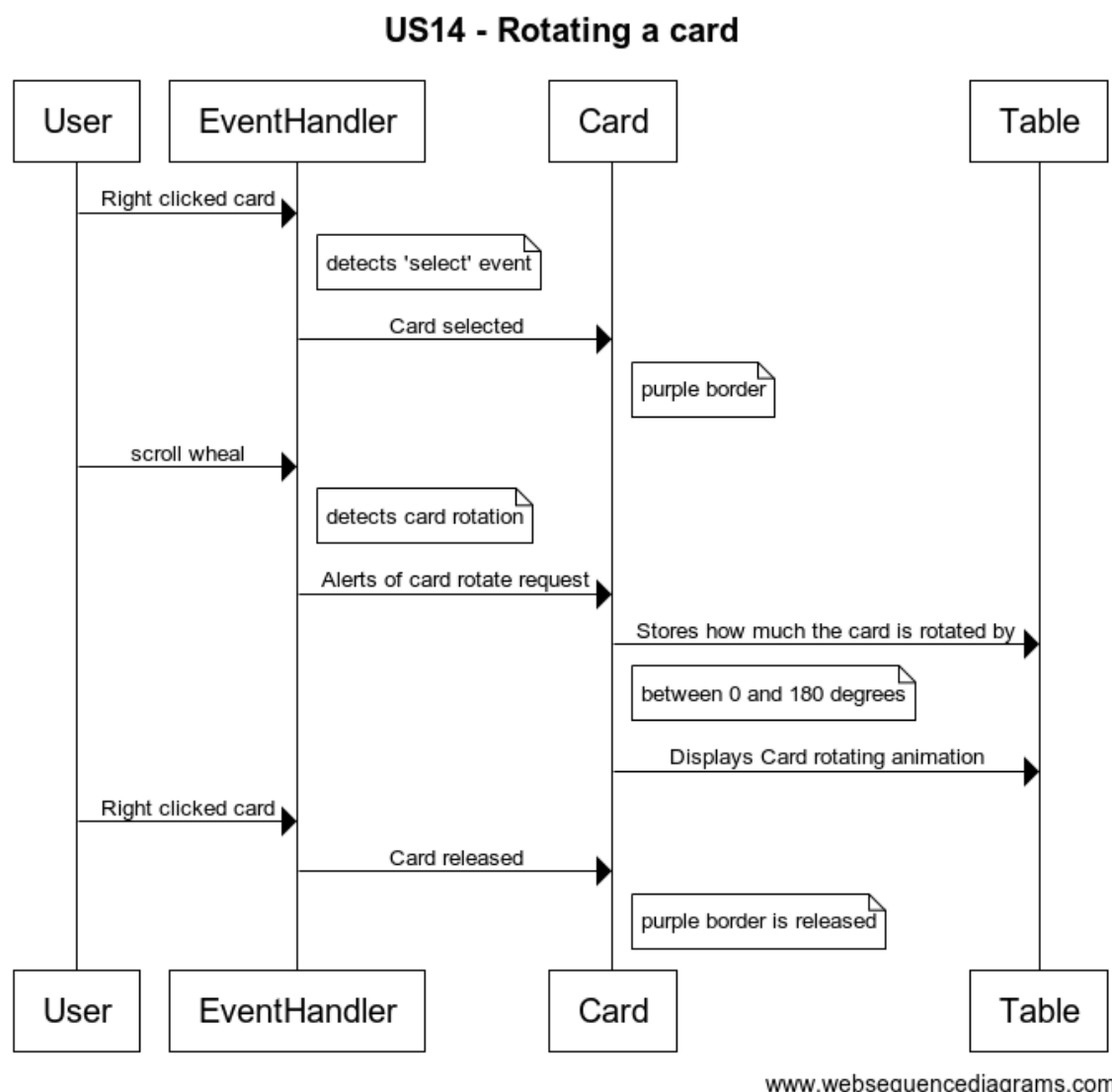


Figure 19: US14 Sequence Diagram

The sequence diagram above shows how a user rotates a selected card. When the user clicks on a card, the system recognises this interaction as an event. This prompts the system to create a rotate transition, which is configured with key parameters like duration, rotation axis, and angle. Once the transition is set up, it's played to animate the card flipping along the Y-axis. After the animation completes, the card's rotation is reset, ensuring it's ready for future interactions.

```
1 BEGIN Rotate
2   SET card to be faceDown
3
4   IF RotateOption is available:
5     CREATE rotateTransition
6     CREATE ImageChangeTransition
7     CREATE ParallelTransition using rotateTransition AND
      ImageChangeTransition
8
9     APPLY ParallelTransition ON cards
10
11    CREATE 5-second delay
12
13    CREATE SequentialTransiton using ParallelTransition AND delay
      AND ParallelTransition
14
15    PLAY SequentialTransiton
16  ENDIF
17
18  SET Card to be returned faceDown
19 END Rotate
```

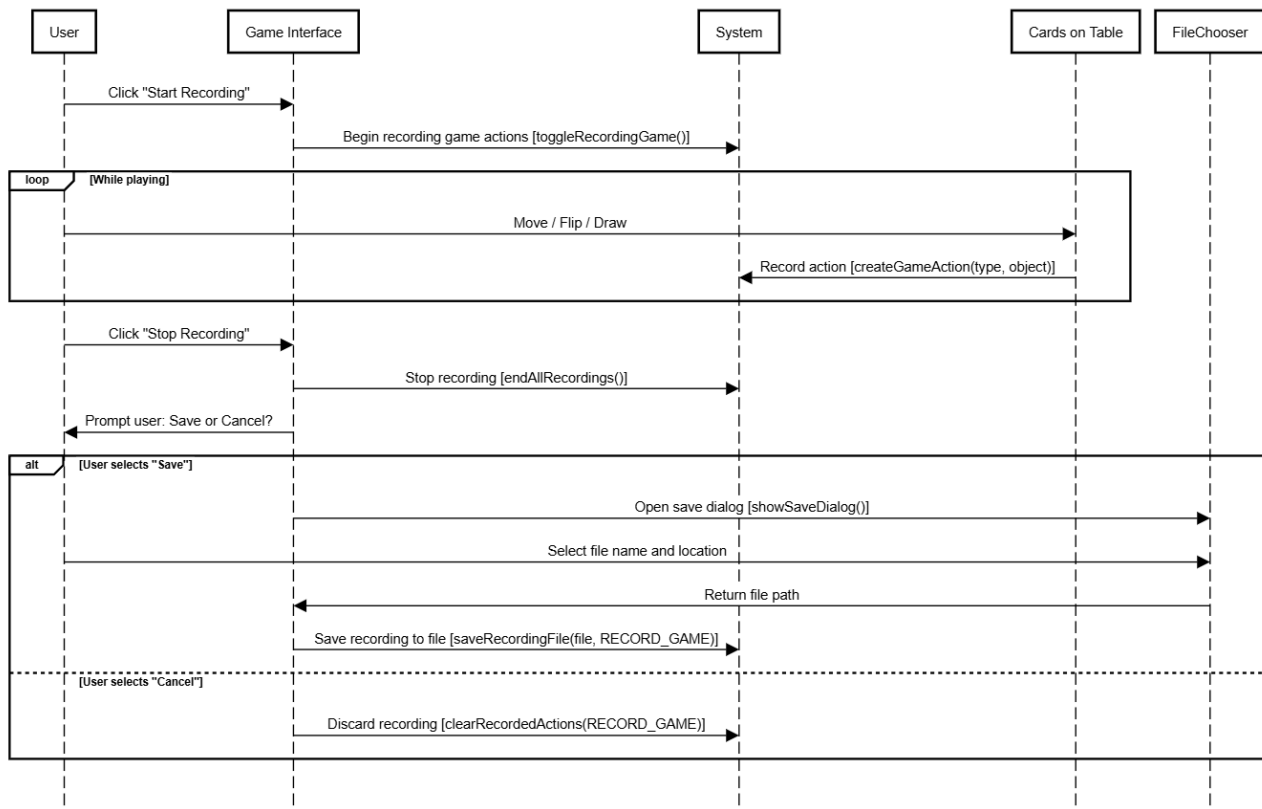
US15: Record Games

Figure 20: US15 Sequence Diagram

The sequence diagram outlines the interactions involved in recording a game session. It begins when the user selects the option to start recording from the game interface, typically by clicking a “Start Recording” button. This triggers the system to enable recording mode via a method like `toggleRecordingGame()`. From that moment onward, all gameplay actions, such as moving cards, flipping them, or drawing from a deck, are captured. Each time the user performs an action, the associated card or table object notifies the system, which creates and stores a `GameAction` using `createGameAction(type, object)`. This loop continues until the user explicitly stops the recording by clicking “Stop Recording”. At that point, the system prompts the user with a dialog box asking whether to save or discard the recording. If the user chooses to save, a `FileChooser` dialog is opened (`showSaveDialog()`), allowing the user to choose a filename and location for the recording. The system then writes the list of recorded actions to the selected file using `saveRecordingFile(file, RECORD_GAME)`. Alternatively, if the user cancels, the system discards the recorded actions using `clearRecordedActions(RECORD_GAME)`, and nothing is saved. This flow ensures that users can preserve game sessions for future playback, or abandon them entirely if no longer needed.


```
1 // When recording button is pressed
2 createNewRecordingThread()
3
4 // After any action is taken in game
5 GameAction gameAction = createActionFromEvent()
6 actionQueue.add(gameAction)
7
8 // To end a dealing layout recording (same button as starting one)
9 GameAction endAction = createActionEndSignal(shouldSave)
10 actionQueue.add(endAction)
11
12 // In the recording thread
13 recording = true
14 WHILE (recording)
15     // thread safe, waits for update
16     action = getActionQueue().take()
17
18     // If it should end, end
19     IF (isEndSignal(action))
20         IF (action.shouldSave())
21             saveRecordingQueueToFile()
22         ENDIF
23         recording = false
24     ENDIF
25
26     // If the action is something that works incrementally like
27     // rotating, it merges it with the
28     // previous action so you dont have an action for each degree of
29     // rotation of the same card
30     action = checkMergeAction(action)
31
32     // If only recording a dealing layout, ignore irrelevant actions
33     IF (!isCardAction(action) AND !isRecordingAll)
34         continue;
35     ENDIF
36
37     // Push it to a queue to be saved
38     recordingQueue.add(action)
39 NEXT
```

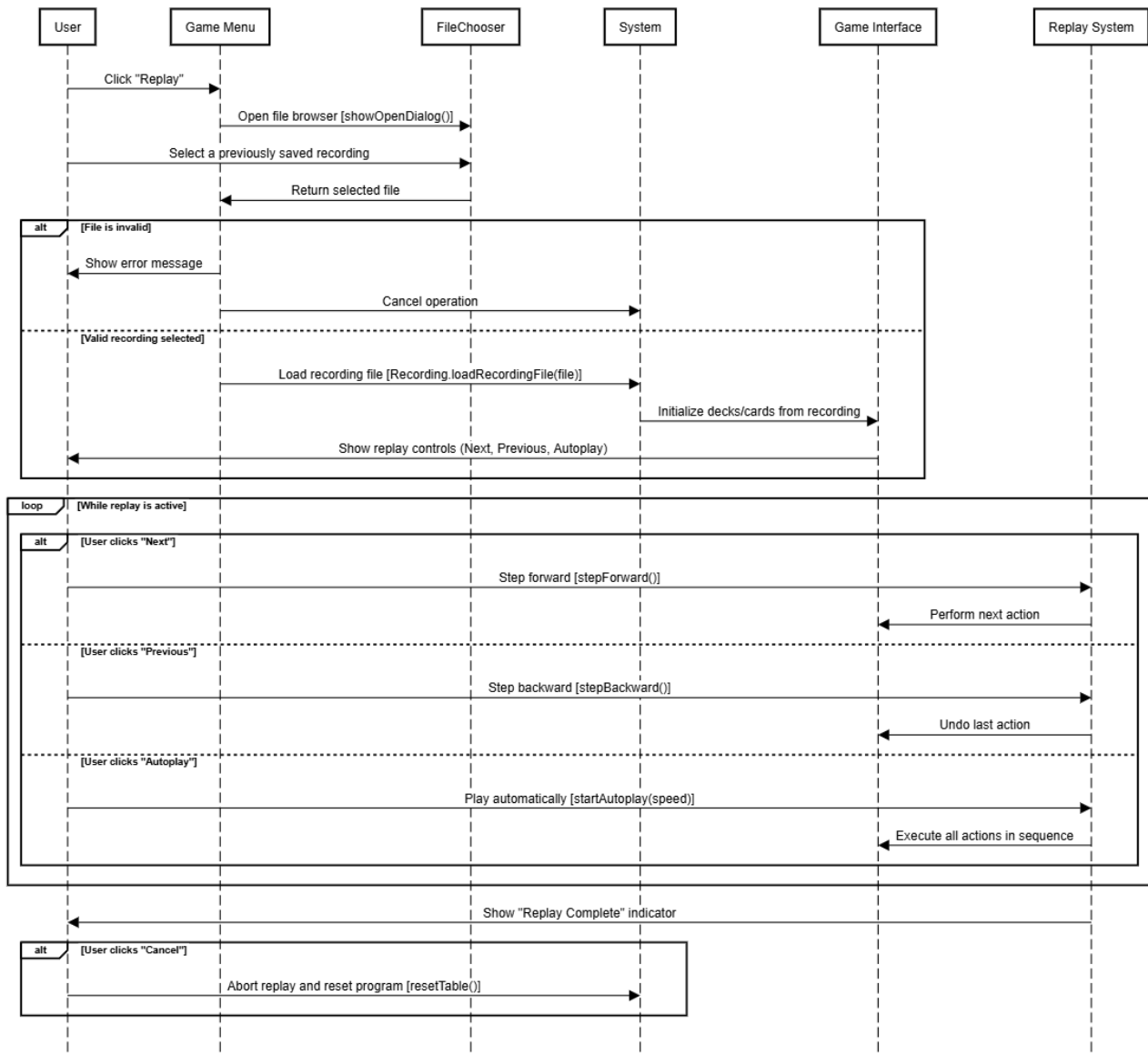
US16: Replay Games

Figure 21: US16 Sequence Diagram

The sequence diagram illustrates how the user interacts with the system to replay a previously recorded game. The process begins when the user selects the “Replay” option from the game’s main menu, which prompts the system to open a file browser dialog (via a FileChooser) where the user can navigate to and select a previously saved recording. If the file is invalid, for example if it’s not a proper recording or is corrupted, the system displays an error message and cancels the operation. However, if a valid file is selected, the system loads the recording using `Recording.loadRecordingFile(file)` and initializes the game interface, setting up the decks and cards as they were when the recording began. Replay controls are then displayed, allowing the user to step through the game manually or start automatic playback. The user can press “Next” to step forward one move at a time, or “Previous” to step backward, with each move executed or undone through the game interface. Alternatively, the user may choose “Autoplay,” which plays the entire recording at a predefined speed (`startAutoplay(speed)`), automatically executing each move. Once the replay is complete, the system displays a visual indicator to notify the user. At any point during the replay, the user can cancel the operation, which stops the playback and resets the game state using `resetTable()`.

```

1 deck = card_deck
2 IF recording file exists
3     get game recording layout file;
4 ELSE display error
5
6 //play whole game recording
7 WHILE recorded action available
8     parse recorded action
9     follow instruction;
10 NEXT
11
12 display done message

```

US17: Snap to Grid

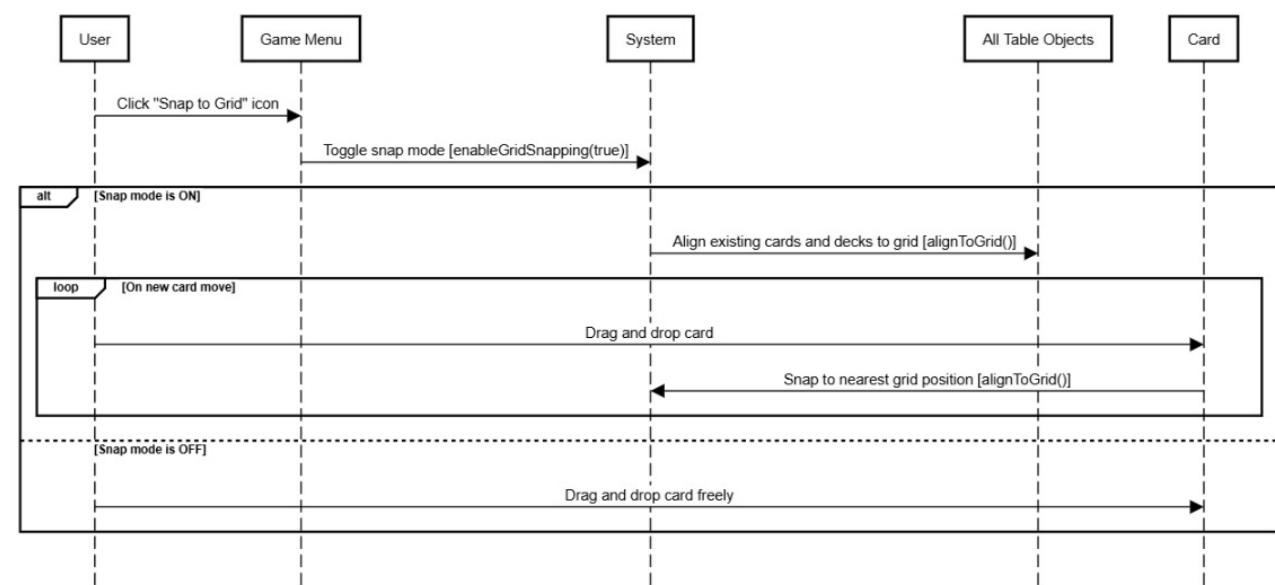


Figure 22: US17 Sequence Diagram

The sequence diagram for US17 includes both the activation of snap-to-grid mode and the alignment of all current cards and decks. When the user clicks the “Snap to Grid” icon from the game menu, the system toggles a flag to enable snap mode e.g., `enableGridSnapping(true)`. The system scans the table and calls `alignToGrid()` on all currently placed objects, repositioning them to the nearest grid-aligned coordinates. This ensures the entire table appears structured when the feature is activated. From that point, whenever the user drags or drops a card, the system intercepts the movement and snaps the card to the nearest grid cell. If the user disables snap mode, the system stops intercepting movement and cards can be placed freely again.

```

1 FUNCTION snapToGrid(card)
2     x = getCardX(card)
3     y = getCardY(card)
4     rotation = getCardRotation(card)
5
6     nearestX = findClosestGridX(x)
7     nearestY = findClosestGridY(y)
8

```

```

9      IF rotation >= -45 AND rotation <= 45 THEN
10         newRotation = 90
11     ELSE
12         newRotation = 0
13     ENDIF
14
15     updateCardPosition(card, nearestGridX, nearestGridY)
16     updateCardRotation(card, newRotation)
17 END FUNCTION
18
19 // On enabling Snap To Grid
20 FOR EACH card IN Cards
21     snapToGrid(card)
22 ENDFOR
23 drawGridLines(cardWidth)
24
25 // Snap cards to grid while the feature is enabled
26 WHILE snapToGridIsEnabled
27     IF cardHasMoved THEN
28         snapToGrid(movedCard)
29     ENDIF
30 ENDWHILE
31
32 // On disabling Snap To Grid
33 removeGridLines()

```

US18: Sound Effects

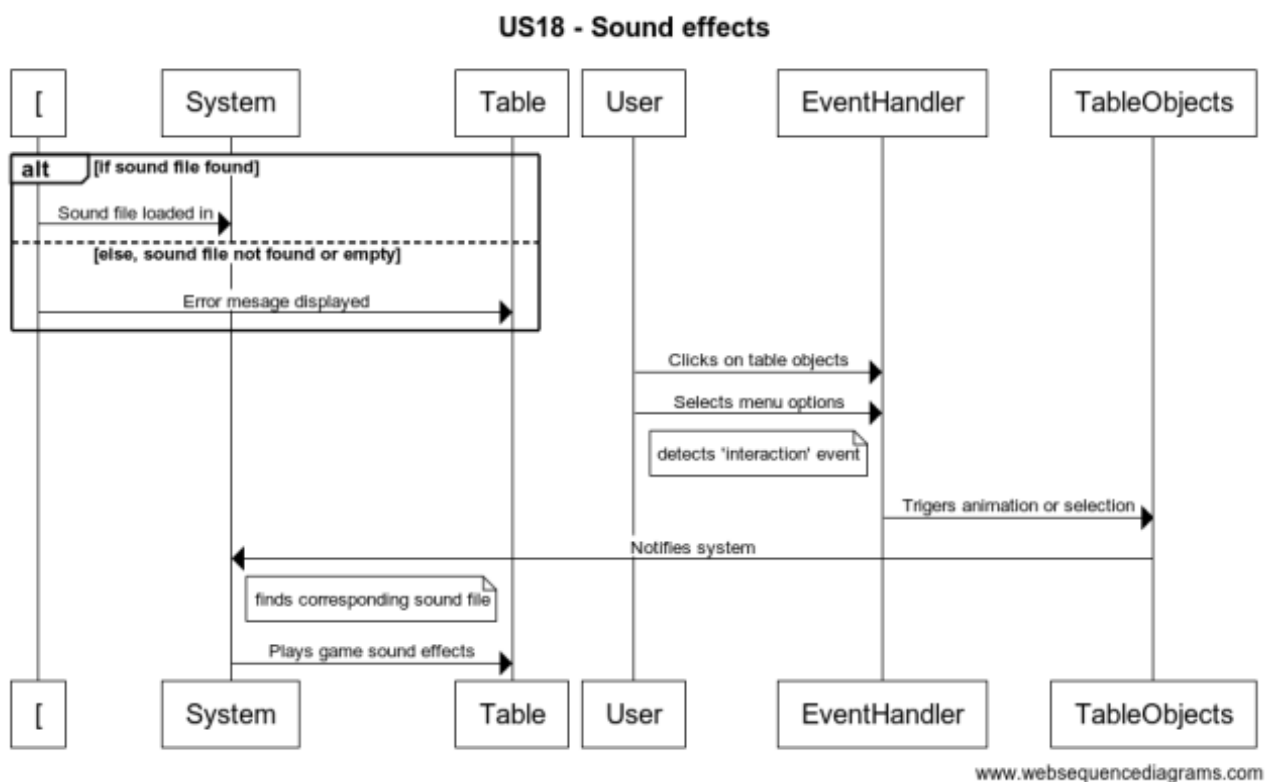


Figure 23: US18 Sequence Diagram

The sequence diagram above illustrates how sound effects are handled when a user interacts with table

objects. At startup, the system attempts to load a sound file; if the file is found, it is loaded into memory, otherwise, the table displays an error message. When the user clicks on any table elements or selects options from their menu, the event is detected as an interaction, and the appropriate button's event handler checks if it triggers an animation or selection, some kind of action, so that it can then know if it should notify the system, which searches for and plays the corresponding sound effect to match the interaction.

```
1 BEGIN PlaySoundOnCardClick
2     LOAD sound file into an AudioClip object
3
4     IF sound file fails to load THEN
5         DISPLAY error message
6     EXIT function
7 END PlaySoundOnCardClick
8
9 DEFINE action to be done on card click
10 CONFIGURE card object so that when clicked, it plays the sound effect
```

4.2 State diagram

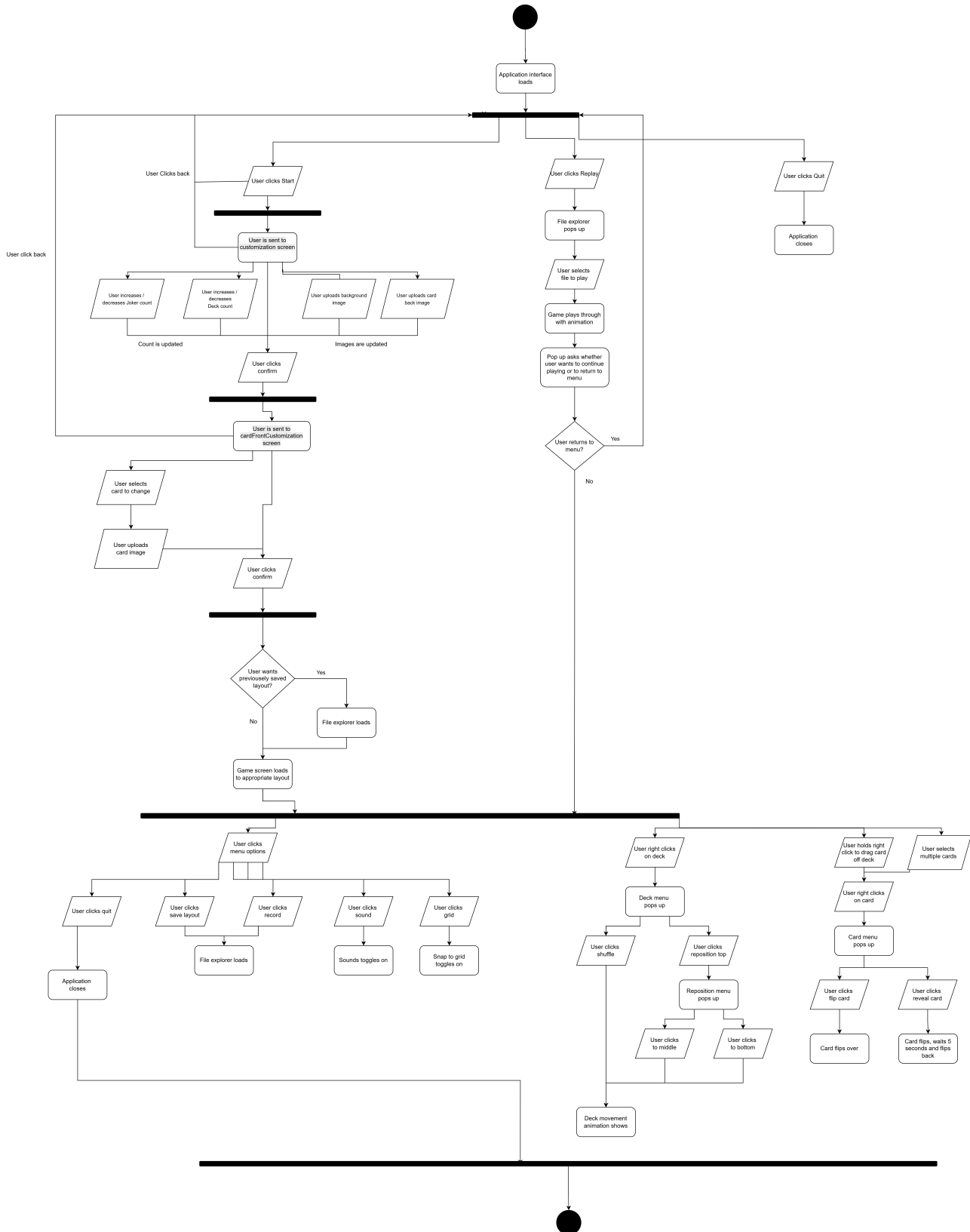


Figure 24: State Diagram

This state diagram represents the user interaction flow within the card-based application interface. It outlines the application's behaviour from initial launch through various interactive states, including customization, gameplay, and termination. Upon entering the application, users are presented with a main interface that branches into several functional areas. These areas include customization options, file handling, and game playing.

The customization process allows users to tailor the visual and structural elements of the gameplay environment, such as choosing their own images for cards and selecting number of jokers wanted. File handling provides a mechanism for loading custom images or previously saved content. When updating images, once file selection is complete, the user is shown the screen they were previously on again, but updated to display their image as required. The gameplay state supports a range of user interactions, including modifying card layouts, engaging with individual or multiple cards, and accessing game menu options. The game menu options include the ability to toggle sound, access replay, or exiting the application. Users may return to prior states, such as the customization screen, from various points within the workflow.

Overall, the diagram captures the dynamic flow between user actions and application responses, highlighting the structure and interactive capabilities of the system without being bound to any single path or sequence. It provides a blueprint for understanding the user experience and the logical transitions that govern application behaviour.

REFERENCES

- [1] *Software Engineering Group Projects* UI Prototype Design Specification. N Perry, F Sasso, N Spence, R Fraser, A Al-Samarrai, J Ball, W Shipp, P Sharma DS1 2.0 Release.
- [2] *Software Engineering Group Projects* Design Specification Standards (CS22120). C.W. Loftus SE.QA.05. 3.4 Release.
- [3] *Software Engineering Group Projects* Product Backlog (CS22120). C.W. Loftus Product Backlog 1.4 Release.

DOCUMENT HISTORY

Version	Issue No.	Date	Changes made to Document	Changed by
0.1	#49	25/02/2025	Added First draft Package diagram	N.S.
	#50		Added system diagram	N.P.
	#52		Added mapping US to classes	R.F., F.S.
0.2	N/A	26/02/2025	Created skeleton for document	R.F.
0.3	#86	11/03/2025	Added introduction paragraph	R.F.
	#127		Updated package diagram	N.S.
0.4	#35	12/03/2025	Reformatted L ^A T _E X version	N.P.
0.5	#177	18/03/2025	Added US1 design diagram and description	R.F.
	#178	18/03/2025	Added US2 design diagram and description	R.F.
	#179	18/03/2025	Added US3 design diagram and description	R.F.
	#180	18/03/2025	Added US4 design diagram and description	R.F.
	#181	18/03/2025	Added US5 design diagram and description	F.S.
	#182	18/03/2025	Added US6 design diagram and description	F.S.
	#184	18/03/2025	Added US8 design diagram and description	N.S.
	#186	18/03/2025	Added US10 design diagram and description	A.A.
	#120	18/03/2025	Added US13 design diagram and description	R.F.
0.6	#54	19/03/2025	Added pseudocode for US1	R.F.
	#61	19/03/2025	Added pseudocode for US2	R.F.
	#62	19/03/2025	Added pseudocode for US3	N.P.
	#63	19/03/2025	Added pseudocode for US4	J.B.
	#64	19/03/2025	Added pseudocode for US5	R.F.
	#65	19/03/2025	Added pseudocode for US6	A.A.
	#66	19/03/2025	Added pseudocode for US7	F.S.
	#67	19/03/2025	Added pseudocode for US8	A.A.
	#68	19/03/2025	Added pseudocode for US9	F.S.
	#69	19/03/2025	Added pseudocode for US10	F.S.
	#70	19/03/2025	Added pseudocode for US11	F.S.
	#71	19/03/2025	Added pseudocode for US12	N.S.
	#72	19/03/2025	Added pseudocode for US13	P.S.
	#73	19/03/2025	Added pseudocode for US14	N.P.
	#74	19/03/2025	Added pseudocode for US15	N.S.
	#75	19/03/2025	Added pseudocode for US16	P.S.
	#76	19/03/2025	Added pseudocode for US17	W.S.
	#77	19/03/2025	Added pseudocode for US18	P.S.
0.7	#183	25/03/2025	Added US7 design diagram and description	N.P.
	#187	25/03/2025	Added US11 design diagram and description	N.S.
	#188	25/03/2025	Added US12 design diagram and description	N.S.
0.8	N/A	25/03/2025	Added system diagram description	W.S.
0.9	#87	01/04/2025	Added system diagram captions	W.S.
	#185	01/04/2025	Added US9 design diagram and description	P.S.
	#189	29/05/2025	Added US14 design diagram and description	F.S.
	#160	01/04/2025	Added US15 design diagram and description	A.A.
	#190	01/04/2025	Added US16 design diagram and description	A.A.
	#162	01/04/2025	Added US17 design diagram and description	A.A.
	#56	01/04/2025	Added state diagram	R.F.
	#219	01/04/2025	Added state diagram description	R.F.
	#186	01/04/2025	Added US11 design diagram and description	R.F.
1.0	#210	29/04/2025	Added US18 design diagram and description	F.S.
1.1	N/A	29/04/2025	Added system diagram description	W.S.
	#88	30/04/2025	Updated package diagram	N.S.
	#51	06/05/2025	Class diagrams	F.S.

Version	Issue No.	Date	Changes made to Document	Changed by
1.2	N/A	07/05/2025	Update pseudocode and diagram descriptions	N.P.
	#223	07/05/2025	Added class descriptions	N.S., A.A., W.S., P.S.
	N/A	08/05/2025	Final cleanup and proof reading	N.S.